

Centro de Investigación y de Estudios Avanzados
del Instituto Politécnico Nacional
Unidad Guadalajara

Auto-diseño de algoritmos en el marco de Sistemas de Auto-Ingeniería (SES)

Tesis que presenta:

Miguel Angel Salazar Martinez

para obtener el grado de:

Maestro en Ciencias

en la especialidad de:

Ingeniería Eléctrica

Director de Tesis

Dr. Mario Angel Siller González Pico

Guadalajara, Jalisco Agosto de 2026

Auto-diseño de algoritmos en el marco de Sistemas de Auto-Ingeniería (SES)

Tesis de Maestría en Ciencias Ingeniería Eléctrica

Por:

Miguel Angel Salazar Martinez

Ingeniero en Sistemas Computaciones
Instituto Tecnológico de Ocotlán 2019-2023

Becario de SECIHTI, expediente no. 245700008

Director de Tesis

Dr. Mario Angel Siller González Pico

Centro de Investigación y de Estudios Avanzados del Instituto Politécnico Nacional
Unidad Guadalajara, Agosto de 2026

Resumen

Abstract

Dedicatoria

A mi madre por ser mi ejemplo a seguir. Agradezco infinitamente tu amor y apoyo incondicional. A mi hermana Saidy y mi hermano Diego, por impulsarme a seguir adelante y ser mi espacio seguro. A mi abuela por enseñarme lo que es ser un buen ser humano. A la vida, por permitirme llegar tan lejos.

Agradecimientos

A todos los que me apoyaron en el camino y me ayudaron en esta travesía del posgrado.
A mi asesor de tesis, el Dr. Mario Siller, por todo el apoyo durante el desarrollo del plan de estudios.
A mi querida familia, a mi madre, a mi padre, a mis hermanos, a mis abuelos, a mis tíos y primos que tanto me han apoyado.
Al grupo de redes de CINVESTAV Guadalajara, que más que grupo, es una familia.
A los doctores del área de computación, por compartir su conocimiento, consejos y, sobre todo, por cultivar la tenacidad. Esa que no aparece en la rubrica, pero que resulta indispensable para llegar hasta aquí.
A todos mis amigos (incluyendo los compis y roomies) especialmente a los miembros del laboratorio de computación, por esas largas charlas, orientación, momentos de calidez y de recreación.
A todos mis maestros, quienes me educaron en toda mi vida académica y nunca desistieron al enseñarme.
Entre ellos destaco a la profesora Aurora Berenice Navarro Núñez, por darme el apoyo que necesite al momento de ingresar a este posgrado.
A todos aquellos que creyeron en mí.
Finalmente, quiero agradecer a la SECIHTI por el apoyo económico brindado, con el cual se logró concluir esta maestría.
Muchas Gracias a todos.

Quiero aclarar que no nombré a todos los involucrados, no por falta de gratitud, sino porque en extensión, los agradecimientos serian inmensos.

Índice general

Resumen I

Abstract II

Dedicatoria III

Agradecimientos IV

1. Introducción 1

1.1. Planteamiento del Problema 2
1.2. Motivación 3
1.3. Objetivos 4
1.3.1. Objetivo General 4
1.3.2. Objetivos Específicos 4
1.4. Hipótesis 4
1.5. Estructura del documento de la Tesis 5

2. Marco Teórico 6

2.1. Ingeniería de Software y el ciclo de vida de los sistemas 6
2.2. Sistemas de Auto-Ingeniería 7
2.2.1. Fases de Operación y Mantenimiento (O & M) y Evolución 8
2.2.2. Actividades de Auto-O & M (Self-O & M) y Auto-Evolución (Self-Evolution) 8
2.2.3. Punto de Quiebre / Breaking Point 8
2.2.4. Elasticidad del Sistema / System Elasticity 9
2.2.5. Metamorfosis / Metamorphosis 9
2.2.6. Gestión de Cambios de Requerimientos / Requirements Change Management (RCM) 9

2.2.7. Meta-Arquitectura del Marco de Sistemas de Auto-Ingeniería 10
2.2.7.1. Módulos Estructurales del Ether 10
2.2.7.2. Operacionalización Mediante un Sistema Multiagente (SES Ether MAS) 11
2.3. Actividades de diseño 12
2.3.1. Diseño de Arquitectura 12
2.3.2. Diseño de Interfaces 13
2.3.3. Diseño de Componentes 13
2.3.4. Diseño de Base de datos 13
2.3.5. Diseño de Algoritmos 14
2.4. Sistemas Multiagentes (MAS) 14
2.4.1. Paradigmas de coordinación y comunicación 15
2.4.2. Comunicación por pizarra o tablero (Blackboard Communication) . 16
2.5. Modelos grandes de lenguaje (LLM) 16
2.5.1. Principios de inferencia y generación de texto en modelos de lenguaje 16
2.5.2. Capacidades de los LLMs en la comprensión, diseño y generación de código 17
2.5.3. Limitaciones actuales: Alucinaciones, ventanas de contexto y deficiencias en el razonamiento algorítmico determinista 17
2.6. Árbol de búsqueda de Monte-Carlo (MCTS) 18
2.7. Ontología como representación de conocimiento 19
2.7.1. Definición y estructura de las ontologías computacionales (clases, atributos, relaciones) 19
2.7.2. Lenguajes de modelado ontológico 20
2.7.3. Modelado semántico para la taxonomía y clasificación de problemas algorítmicos 21

3. Estado del Arte **23**
3.1. Conceptualización del Auto-Diseño 23
3.2. Automatización del Diseño 24
3.2.1. Auto-diseño basado en Inteligencia Artificial y Aprendizaje 24
3.2.2. Autodiseño bio-inspirado, de reúso y búsqueda 25
3.2.3. Autodiseño basado en modelado de agentes 25
3.3. Conceptos Emergentes y Mecanismos de Control en Autodiseño 26
3.4. Comparación de Trabajos 26

4. Estado del Arte y Actividades de Auto-Diseño **27**
4.1. Automatización de las Actividades de Diseño de Software 27
4.1.1. Actividades de Diseño Arquitectónico (Macro-nivel) 27
4.1.2. Actividades de Diseño de Componentes e Interfaces (Meso-nivel) . . 28
4.1.3. Actividades de Diseño de Estructuras de Datos (Micro-nivel de Almacenamiento) 28
4.1.4. Actividades de Diseño de Algoritmos y Heurísticas (Micro-nivel Lógico) 29
4.2. Dimensión Transversal: Soporte de Ejecución y Auto-Evolución 29
4.3. Síntesis Comparativa y Declaración del Vacío de Investigación 30
4.3.1. Declaración del Vacío de Investigación (Research Gap) 31

5. Propuesta **32**
5.1. Metodología 32
5.1.1. Revisión Sistemática de literatura 32
5.1.1.1. Formulación preguntas de investigación 32
5.1.1.2. Estrategia de búsqueda 33
5.1.1.3. Criterios de Inclusión y de exclusión 37
5.1.1.4. Proceso de selección de estudios 39
5.1.1.5. Extracción de datos 41
5.1.1.6. Resultados 42
5.1.1.7. Discusión 42
5.2. Propuesta de un Sistema Multi-Agente (MAS) para el Auto-Diseño 43
5.2.1. Formulación General del Sistema 43
5.2.1.1. Definición del Entorno y Modelo de Auto-Diseño 43
5.2.1.2. Formalización Matemática del MAS 43
5.2.2. Especificación del Sistema (System Specification) 43
5.2.2.1. Jerarquía de Metas (System Goals) 43
5.2.2.2. Interfaz con el Entorno (Percepts y Actions) 43
5.2.2.3. Escenarios de Uso Principales (Scenarios) 44
5.2.3. Diseño Arquitectónico (Architectural Design) 44
5.2.3.1. Identificación y Justificación de Agentes 44
5.2.3.2. Estructura Global del Sistema (System Overview) 44
5.2.3.3. Protocolo de Negociación y Coordinación 45
5.2.4. Diseño Detallado BDI (Detailed Design) 45

5.2.4.1. Estructura BDI Interna de los Agentes 45
5.2.4.2. Caso de Estudio de Caja Blanca: Agente de Algoritmos (A_{alg}) 45
5.2.5. Análisis de Propiedades del Modelo de Agentes 46

6. Experimentación y Análisis de Resultados **47**
6.1. Análisis de selección automática de algoritmos 47
6.1.1. Análisis de la arquitectura: auto-diseño 47
6.2. Experimentación 47
6.2.1. Caso de estudio: nuevo problema, nueva solución 47
6.2.2. Experimento: Identificación del Problema 47
6.2.3. Experimento: Identificación del Algoritmo 47

7. Conclusiones y Trabajo Futuro **48**
7.1. Conclusiones 48
7.2. Objetivos logrados 48
7.3. Trabajo a futuro 48

Referencias Bibliográficas **49**

A. Código Fuente Relevante **55**

Índice de figuras

2.1. Diagrama del proceso de diseño inspirado en el diagrama 2.5 de Ingenieria de software (10ma edicion) de Sommerville [6].(Diagrama elaborado en Draw.io) 12

5.1. Revisión del flujo de trabajo basado en PRISMA.(Diagrama elaborado en Draw.io) 40

5.2. Proceso de selección de artículos con resultados cuantitativos por etapa de cribado. (Diagrama elaborado en Draw.io) 41

Índice de tablas

3.1. Matriz comparativa de enfoques de automatización y artefacto de software. 26

4.1. Matriz comparativa de enfoques de automatización según actividades de diseño y brecha de Self-Design. 30

5.1. Actividades de diseño y sus correspondientes artefactos de diseño. 33
5.2. Ejemplo de una consulta de búsqueda completa (Scopus) 35
5.3. Resumen de las consultas en Scopus (Obviando los filtros temporales y de dominio para mantener la brevedad). 36
5.4. Resumen de los criterios de inclusión y exclusión 38
5.5. Especificación BDI de los agentes del MAS propuesta. 45

Capítulo 1

Introducción

El desarrollo de sistemas de software modernos requiere la ejecución coordinada de múltiples actividades de ingeniería de software, desde el análisis de requerimientos, diseño, implementación y pruebas, hasta las fases de operación, mantenimiento y eventual reemplazo de los sistemas; A lo largo de las últimas décadas, diversos estudios han evidenciado que las actividades de mantenimiento, actualización y evolución de artefactos de software constituyen una de las etapas más costosas del ciclo de vida del software, tanto en términos de capital humano como de tiempo de desarrollo y recursos financieros [1], [2]. Esta situación ha motivado la búsqueda de enfoques capaces de reducir la dependencia de la intervención humana en dichos procesos y automatizar las tareas de evolución del software. En este contexto surge la teoría de los Sistemas de Auto-Ingeniería (*Self-Engineering Systems*, SES), la cual propone un marco conceptual y una meta-arquitectura que permiten a los sistemas ejecutar modificaciones autónomas sobre sí mismos, habilitando procesos de auto-evolución o metamorfosis [3]. **Su propósito fundamental es gestionar y ejecutar su propio proceso de ingeniería más allá del ciclo de vida convencional, minimizando o incluso eliminando la necesidad de intervención humana** [4].

Una de las actividades más relevantes dentro de la ingeniería de software es el diseño, esta actividad es la encargada de traducir los requerimientos funcionales, no funcionales y los requisitos de dominio en especificaciones técnicas detalladas y sus respectivos artefactos de diseño para la posterior implementación y ejecución del sistema. En el contexto de los SES [4], se persigue la automatización de las actividades que conforman el proceso tradicional de diseño, es decir, **dotar al sistema de capacidades de auto-diseño**. Esto implica que el sistema no solo reaccione ante fallos, sino que además pueda reconfigurar su lógica interna de forma válida, segura y coherente con las decisiones arquitectónicas

previamente establecidas.

Diversos trabajos han explorado la automatización de actividades específicas del diseño de software; sin embargo, la evidencia reportada en la literatura sugiere que estas iniciativas suelen enfocarse en etapas particulares del proceso de diseño y rara vez abordan su integración de manera integral todas las actividades del mismo. Esto se analiza a profundidad en el capítulo 3.

En respuesta a esta problemática, **la presente investigación propone una meta-arquitectura basada en un sistema multiagente para la generación autónoma de artefactos de diseño.** Este tipo de arquitectura establece un marco de referencia y un conjunto de reglas de composición que permiten coordinar múltiples configuraciones bajo un modelo unificado [5]. Como fundamento conceptual, esta propuesta adopta las actividades de diseño de software descritas por Sommerville [6]. Bajo esta perspectiva la propuesta está conformada por cuatro agentes especializados, encargados respectivamente del diseño arquitectónico, el diseño de componentes e interfaces, el diseño de estructuras de datos y el diseño algorítmico. En conjunto, estos agentes colaboran para construir una especificación progresivamente refinada a partir de los requerimientos del sistema, mediante sus procesos individuales y mecanismos de coordinación que permiten integrar sus resultados en una solución de diseño final.

No obstante, como prueba de concepto el desarrollo e implementación se concentran en el agente responsable del diseño algorítmico.

Las principales contribuciones de esta investigación son las siguientes:

1. Una caracterización del estado del arte sobre diseño automatizado de software mediante una revisión sistemática de literatura.

2. Una meta-arquitectura basada en sistemas multiagente para la generación autónoma y coordinada de artefactos de diseño de software.

3. Un modelo de agente de diseño algorítmico fundamentado en la representación ontológica del conocimiento y la asociación problema-solución para la selección automática de algoritmos.

1.1. Planteamiento del Problema

Durante el desarrollo, operación y mantenimiento de un sistema de software, este experimenta cambios en sus requerimientos que pueden derivar en la creación o modificación

de distintos artefactos de diseño [7]. La actividad de diseño abarca desde la definición de la arquitectura del sistema y la especificación de sus componentes hasta la selección de estructuras de datos, el diseño de algoritmos y otros artefactos con distinto nivel de detalle, como diagramas y casos de uso [8]. Bajo la teoría de los Sistemas Evolutivos de Software (SES) [4], se busca que estas actividades puedan realizarse sin intervención humana. Sin embargo, aún no existe un mecanismo que automatice la actividad de diseño dentro de dicha teoría. A partir de este planteamiento surgen los siguientes cuestionamientos:

1. ¿Es posible dotar a un sistema de un mecanismo de diseño completamente automatizado?

2. Ante cambios en los requerimientos de un sistema en operación, ¿cómo pueden generarse nuevos artefactos de diseño o modificarse los existentes y, en caso necesario, sustituirse de manera consistente?

3. Cuando se produce un nuevo sistema mediante un proceso de metamorfosis, ¿en qué medida es posible reutilizar los artefactos y el conocimiento previamente generados?

Los artefactos resultantes deben permitir documentar y comunicar las decisiones de diseño, facilitar su validación y mantener la trazabilidad entre los requerimientos, la arquitectura, los componentes y la implementación.

1.2. Motivación

La presente investigación surge de la necesidad de avanzar hacia sistemas de software capaces de participar activamente en su propia evolución mediante la generación y adaptación de artefactos de diseño. A medida que los sistemas de software incrementan su complejidad, las actividades de mantenimiento suelen incrementar la deuda técnica acumulada (derivado de priorizar el desarrollo) [1]. Esta situación se agrava por el hecho de que los sistemas operan en entornos dinámicos y sujetos a cambios constantes [9], donde la dependencia de la intervención humana para analizar requerimientos, rediseñar soluciones y actualizar artefactos de diseño limita su capacidad de respuesta y adaptación. En este contexto, la automatización de las actividades de diseño de software representa un paso fundamental para habilitar mecanismos de razonamiento y toma de decisiones sobre la estructura y comportamiento de los sistemas. La integración de agentes especializados y mecanismos de representación explícita del conocimiento permite establecer las bases para que futuras generaciones de software puedan modificar y justificar sus decisiones de diseño

de manera consistente. Finalmente, esta investigación busca contribuir al desarrollo de capacidades de autodiseño que funcionen como habilitador para la evolución y metamorfosis de sistemas SES [3], buscando erradicar la dependencia del mantenimiento manual.

1.3. Objetivos

1.3.1. Objetivo General

Desarrollar una meta-arquitectura basada en agentes para el diseño automatizado de software, capaz de abstraer y coordinar las distintas actividades de diseño con el fin de facilitar la evolución autónoma de artefactos de software ante cambios en los requerimientos.

1.3.2. Objetivos Específicos

- Sintetizar el estado del arte del diseño automatizado de software y la selección algorítmica autónoma mediante una Revisión Sistemática de la Literatura (RSL o SLR del ingles Sistematic Literature Review), con el fin de identificar tendencias, limitaciones y oportunidades de investigación.
- Diseñar una meta-arquitectura basada en sistemas multiagente que modele y coordine el proceso de diseño de software, definiendo mecanismos de interacción entre sus distintas actividades.
- Desarrollar un agente de diseño algorítmico fundamentado en representación ontológica del conocimiento, capaz de establecer asociaciones problema-solución para la selección autónoma de alternativas algorítmicas.
- Validar la funcionalidad de la meta-arquitectura propuesta mediante la implementación y evaluación del agente de diseño algorítmico en escenarios experimentales controlados, analizando su capacidad para adaptarse a cambios en los requerimientos y mantener la consistencia de los artefactos de diseño generados.

1.4. Hipótesis

Es posible gestionar de manera autónoma la generación y adaptación de artefactos de diseño de software ante cambios en los requerimientos mediante una meta-arquitectura

basada en agentes especializados, capaz de coordinar distintas actividades de diseño y seleccionar alternativas de solución consistentes con las restricciones y decisiones previamente establecidas.

1.5. Estructura del documento de la Tesis

La presente tesis se estructura en seis capítulos que articulan el desarrollo de la investigación. El capítulo 1 introduce el problema de investigación, la motivación, los objetivos, las hipótesis y el contexto general del estudio. El capítulo 2 presenta los fundamentos teóricos necesarios para comprender el trabajo, abordando conceptos relacionados con los Sistemas de Auto-Ingeniería (SES), la automatización, el diseño de software y la selección algorítmica. A continuación, el capítulo 3 examina el estado del arte mediante una revisión crítica de la literatura, identificando tendencias, enfoques metodológicos, limitaciones y oportunidades de investigación. En el capítulo 5 se describe la meta-arquitectura propuesta, detallando sus fundamentos conceptuales, metodología, estructura y mecanismos de selección automática. Posteriormente, el capítulo 6 presenta su validación experimental a través de casos de estudio, escenarios simulados y el análisis de los resultados obtenidos. Finalmente, el capítulo 7 expone las conclusiones generales, las principales contribuciones del trabajo, sus limitaciones y las líneas de investigación futura derivadas de este estudio.

Capítulo 2

Marco Teórico

El Marco Teórico constituye el cimiento conceptual sobre el cual se erige esta investigación. Su propósito fundamental es articular las teorías, paradigmas y modelos previos que otorgan validez científica a la problemática planteada, sirviendo como mapa orientador para el desarrollo de la propuesta.

2.1. Ingeniería de Software y el ciclo de vida de los sistemas

La Ingeniería de Software es una disciplina de la ingeniería que se interesa por todos los aspectos de la producción de software[7], cuyo propósito principal es la aplicación de principios, métodos y herramientas sistemáticas para el desarrollo, operación y mantenimiento de software de alta calidad. Si bien no existe una única definición ni caracterización universal de este proceso de ingeniería, trabajos de referencia como los de [6], [10] coinciden en que esta disciplina abarca no solo los aspectos técnicos de la construcción de programas. Por su parte, la guía SWEBOK [10] la define como un enfoque sistemático que busca resolver problemas complejos de desarrollo mediante la aplicación de procesos estructurados y estandarizados. En el corazón de la Ingeniería de Software se encuentra el Ciclo de Vida del Desarrollo de Sistemas (SDLC, por sus siglas en inglés *Systems Development Life Cycle*). Este modelo conceptual describe las fases necesarias para llevar un sistema informático desde su concepción inicial, pasando por su diseño y construcción, hasta su implementación, operación y eventual retirada [10]. Tradicionalmente, en los enfoques predictivos o estructurados, el ciclo de vida se organiza en etapas bien definidas:

- **Análisis de Requisitos:** Identificación y documentación de las necesidades del usuario, las características operativas y las restricciones del sistema.
- **Diseño del Sistema y del Software:** Traducción de los requisitos en una arquitectura global y especificaciones detalladas de componentes, bases de datos e interfaces.
- **Implementación (Codificación):** Construcción del código fuente y unidades de programa utilizando los lenguajes y herramientas de desarrollo adecuados.
- **Pruebas (Verificación y Validación):** Detección sistemática de errores para comprobar que el software cumple con su especificación técnica (verificación) y satisface las necesidades reales del usuario en su entorno operativo (validación).
- **Despliegue y Transición:** Puesta en marcha e instalación del sistema en el entorno de producción para los usuarios finales.
- **Mantenimiento y Evolución:** Evolución continua del software para corregir fallos, mejorar el rendimiento o adaptarlo a nuevos entornos tecnológicos o requerimientos cambiantes.
- **Retiro o Reemplazo (Disposición final):** Gestión de la desactivación, archivo de datos y sustitución del sistema al término de su ciclo de vida o tras la entrada de un nuevo sistema que lo releve.

Dependiendo de la naturaleza del proyecto, estas fases pueden adoptarse mediante enfoques predictivos tradicionales (como el modelo en cascada) o metodologías ágiles e iterativas, las cuales permiten una mayor flexibilidad ante los cambios constantes en los requerimientos del entorno [10], [11], [12].

2.2. Sistemas de Auto-Ingeniería

En el trabajo de Perez-Carrillo Et. Al. [3] se busca que los mismos sistemas no terminen el ciclo de su vida en desecho, si no, que sin intervención humana continúen su operación bajo el concepto de elasticidad del sistema o que sufran una metamorfosis dando pie a un nuevo sistema, esta perspectiva se conoce como el marco de los sistemas de auto-ingeniería. Los SES contemplan un proceso muy similar a la ingeniería de software tradicional, es decir, las actividades como lo son ingeniería de requerimientos, diseño, implementación, pruebas, etc.[7] tienen lugar en este marco de trabajo. Este trabajo unicamente esta enfocado en el

diseño del software automatizado, basado en lineamientos presentes en [10]. Por lo tanto el foco de este manuscrito esta puesto en la producción de artefactos de diseño, que fungen como planos del sistema antes de implementación en código.

La teoría SES introduce conceptos que permiten identificar fases y actividades dentro de la vida de estos sistemas [4]:

2.2.1. Fases de Operación y Mantenimiento (O & M) y Evolución

A diferencia de la literatura tradicional que debate si el mantenimiento es un subconjunto de la evolución o viceversa, el marco SES [4] establece las fases de Operación y Mantenimiento (O&M) y Evolución como fases independientes.

- Fase de O&M: Se ejecutan mantenimientos correctivos, preventivos, adaptativos, aditivos y de mejora.
- Evolución por Mantenimiento: Se genera cuando los mantenimientos adaptativos, aditivos o perfectivos provocan cambios en los requerimientos.
- Evolución por Cambio de Problema: Ocurre cuando cambia el problema que el sistema busca resolver, siempre y cuando la arquitectura actual sea capaz de soportar dichos cambios.

2.2.2. Actividades de Auto-O & M (Self-O & M) y Auto-Evolución (Self-Evolution)

Son las actividades fundamentales (descritas previamente) que el sistema ejecuta de manera autónoma tras su despliegue dentro de las fases de O&M y Evolución, respectivamente, de allí el nombre de self, es decir desarrollado por si mismo.

2.2.3. Punto de Quiebre / Breaking Point

Representa el umbral en el cual la arquitectura original del sistema ya no puede satisfacer los nuevos requerimientos. Al alcanzar el **Breaking Point**, se suspenden las actividades de Self-O&M y Self-Evolution, desencadenando el inicio de un nuevo proceso e inicio de ciclo de vida de ingeniería presentado como **metamorfosis** [4].

2.2.4. Elasticidad del Sistema / System Elasticity

Se define como la capacidad del sistema para asimilar requerimientos nuevos o modificados sin la necesidad de construir un nuevo sistema [4].

2.2.5. Metamorfosis / Metamorphosis

Es la fase fundamental del paradigma SES, definida como la **trascendencia del sistema original (Sistema A) hacia un nuevo sistema (Sistema B)** mediante un nuevo proceso de Auto-Ingeniería tras alcanzar un punto de ruptura. Involucra la ejecución coordinada de las actividades SES *Self-*: Self-Requirements, Self-Design, Self-Implementation, Self-Testing* y Self-Deployment* [4].

Se desencadena bajo dos condiciones principales:

- Cuando se requiere un cambio de arquitectura a nivel conceptual (el problema o dominio de aplicación cambió y la arquitectura actual no puede resolverlo).
- Cuando los cambios de requerimientos impactan la arquitectura lógica y esta es incapaz de acomodarlos.

A diferencia de la ingeniería tradicional donde un arquitecto humano toma la decisión, en el marco SES el propio sistema toma la decisión de manera autónoma.

2.2.6. Gestión de Cambios de Requerimientos / Requirements Change Management (RCM)

La estructura de **requerimientos funcionales** se divide en dos componentes principales, la *funcionalidad* se manifiesta a través de las acciones principales, expresadas en verbos, y la *configuración* especifica las condiciones y los parámetros que deben seguirse [4]. En lo que respecta a los **requerimientos no funcionales**, las métricas y los parámetros se tratan como componentes de configuración. En el caso de que el cambio suponga una afectación a la funcionalidad o bien genere un impacto en la arquitectura que la actual si soporte y permita el cambio, este se gestiona mediante actividades de auto-operación y mantenimiento (Self-O&M) y auto-evolución (Self-Evolution). En caso contrario (no se soporta), se provoca una metamorfosis. El procedimiento RCM, adaptado para SES (basado en lo descrito por Sommerville [7]), se presenta de la siguiente manera:

- *Cambio de especificaciones*: Surge cuando el problema que resuelve el sistema ha cambiado, o cuando se solicita un cambio específico. En esta fase, cada solicitud se valida [4].
- *Análisis del Cambio*: Evaluación de la elasticidad del sistema para determinar si el cambio produce una nueva versión es decir continuamos en proceso de evolución o si se alcanzó el punto de quiebre se desencadena una metamorfosis [4].
- *Implementación del Cambio*: Si la arquitectura lo soporta, el sistema evoluciona a una nueva versión; si no lo soporta, se inicia un nuevo proceso de Auto-Ingeniería dando lugar a un nuevo sistema mediante la metamorfosis [4].

2.2.7. Meta-Arquitectura del Marco de Sistemas de Auto-Ingeniería

Un Sistema de Auto-Ingeniería (*SES*) se define por su capacidad para gestionar y llevar a cabo de manera autónoma su propio proceso de ingeniería a lo largo de su ciclo de vida y durante la trascendencia del mismo [4]. El núcleo de este paradigma se fundamenta en la Meta-Arquitectura de Referencia, denominada *Ether*, una abstracción estructural que permanece persistente por encima de la arquitectura central del sistema y se conserva a lo largo de las versiones del mismo (incluyendo nuevos sistemas) [4]. El *Ether* es responsable de orquestar las capacidades de gestión, evolución y metamorfosis descritas anteriormente.

2.2.7.1. Módulos Estructurales del Ether

El *Ether* está conformado por cinco módulos conceptuales coordinados que procesan los eventos de cambio y dictan el flujo del ciclo de vida del sistema [4]:

- **Repositorio RACR (*Requirements, Architectural description and implemented Changes log Repository*):** Almacena las especificaciones funcionales y no funcionales, las descripciones arquitectónicas (p. ej., diagramas UML) y el historial de cambios implementados. En un *Pure SES*, estos registros se generan de forma autónoma, mientras que en un *Hybrid SES* la configuración inicial la proporcionan ingenieros humanos.
- **Manejador de Eventos (*Event Handler*):** Monitorea continuamente las solicitudes de cambio en los requerimientos solicitadas por el entorno, fallos de mantenimiento o cambios en el problema original.

- **Módulo de Gestión de Cambios de Requerimientos (*RCM Module*):** Representa el núcleo operativo del *Ether*. Ejecuta la especificación de cambios, el análisis de impacto y evaluación de la elasticidad arquitectónica para determinar si se ha alcanzado el *Punto de Quiebre*, y la ordenación de la implementación.
- **Módulo de Evolución:** Se activa si la arquitectura actual tolera los cambios sin llegar al *Punto de Quiebre*. Orquesta las actividades de *Self-O&M* y *Self-Evolution* para desplegar una nueva versión del mismo sistema.
- **Módulo de Metamorfosis:** Se activa cuando se alcanza el *Punto de Quiebre*. Define e implementa un nuevo proceso de *Self-Engineering* que concluye en el despliegue de un nuevo sistema con una arquitectura renovada, conservando la abstracción del *Ether*.

2.2.7.2. Operacionalización Mediante un Sistema Multiagente (SES Ether MAS)

Para funcionar sin intervención humana, la meta-arquitectura *Ether* se operacionaliza como un Sistema Multiagente estructurado en cinco roles de agentes especializados [4]:

1. **Event Handler Agent:** Escucha y captura las peticiones de cambio de requerimientos enviadas por los actores desencadenantes.
2. **RACR Manager Agent:** Administra la consulta y actualización del repositorio RACR, estandarizando los formatos de requerimientos y arquitectura.
3. **Analyzer Agent:** Valida la factibilidad de los cambios y evalúa si la elasticidad de la arquitectura soporta la modificación o si se ha llegado al *Breaking Point*.
4. **Change Process Orchestrator Agent:** Genera la especificación técnica de la nueva versión (Evolución) o diseña el nuevo proceso de ingeniería completo (Metamorfosis).
5. **Change Process Executor Agent:** Ejecuta la implementación de los cambios o despliega el nuevo sistema a través de las actividades *Self-**.

2.3. Actividades de diseño

Siguiendo el modelo general de diseño propuesto por Sommerville en las dos ultimas ediciones de *"Software Engineering"* [6], [7], se contemplan las actividades de: *Diseño Arquitectónico, Diseño de Interfaces, Diseño de Componentes y Diseño de Bases de Datos.*

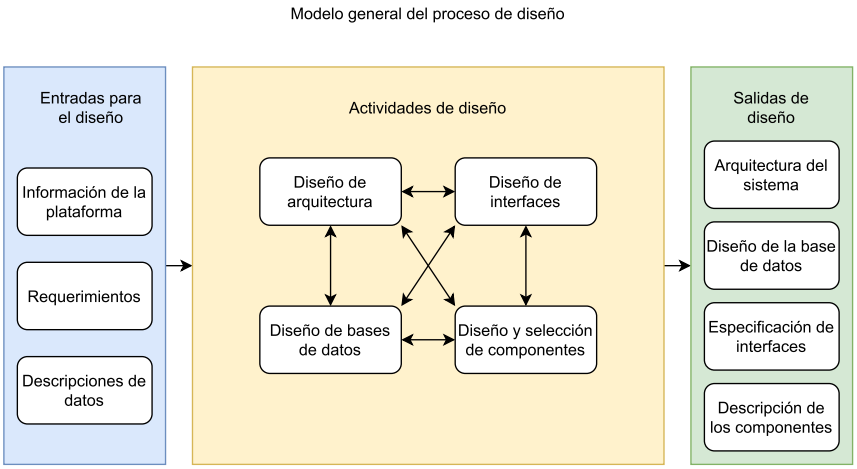


Figura 2.1: Diagrama del proceso de diseño inspirado en el diagrama 2.5 de Ingenieria de software (10ma edicion) de Sommerville [6].(Diagrama elaborado en Draw.io)

Adicionalmente, se incluye de forma explícita la actividad de Diseño de Algoritmos como parte del diseño detallado de componentes. Esta decisión se fundamenta en: La necesidad de especificar con precisión la lógica cuando los algoritmos presentan alta complejidad computacional o requisitos no funcionales críticos (rendimiento, tiempo real, etc.). La recomendación de estándares y guías de referencia como lo son el *"Software Design Descriptions"* (SDD) [13]y la *"Guide to the Software Engineering Body of Knowledge"* [10] reconocen el diseño de algoritmos como una actividad diferenciada dentro del diseño detallado porque facilita la verificación, el análisis de complejidad y la posterior implementación.

2.3.1. Diseño de Arquitectura

Según Sommerville [7], el diseño arquitectónico consiste en identificar la estructura general del sistema, los componentes principales (también denominados sub-sistemas o módulos), sus relaciones y la forma en que se distribuyen. Esta actividad produce una visión de alto nivel que define cómo debe organizarse el sistema para satisfacer los requisitos funcionales y no funcionales. El *SWEBOK* [10] y el estándar *IEEE 1016* [13] refuerzan

esta idea al considerar la arquitectura como la organización fundamental de los elementos de software y sus relaciones.

2.3.2. Diseño de Interfaces

Esta actividad define las interfaces entre los componentes del sistema. *Sommerville* [6] destaca que una vez especificadas las interfaces, los componentes pueden diseñarse y desarrollarse de forma concurrente, ya que cada uno solo necesita conocer el contrato de comunicación (operaciones, parámetros y comportamientos esperados). Esto favorece un bajo acoplamiento y una alta cohesión. El estandar *IEEE 1016* [13] incluye el punto de vista de la interfaz (Interface Viewpoint) como una de las vistas recomendadas para documentar estos contratos.

2.3.3. Diseño de Componentes

En esta etapa se especifica con mayor precisión cómo operará cada uno de los componentes identificados durante el diseño arquitectónico. El diseño de componentes puede adoptar diferentes niveles de detalle según las necesidades del proyecto: desde una descripción funcional sencilla que indique qué debe hacer el componente, hasta la adaptación de un componente reutilizable ya existente, o bien la elaboración de un modelo de diseño detallado que defina su estructura interna, comportamiento y relaciones [6]. El *SWEBOK* sitúa esta actividad dentro del diseño detallado (detailed design), cuyo propósito fundamental es describir cada componente con el nivel de detalle suficiente para que pueda ser construido de manera inequívoca por los desarrolladores. Esto implica definir no solo las responsabilidades del componente, sino también sus interfaces internas, las estructuras de datos que manipula, los algoritmos principales que ejecuta y las restricciones de rendimiento o de calidad que debe satisfacer [10].

2.3.4. Diseño de Base de datos

El diseño de bases de datos consiste en definir las estructuras de datos del sistema y la forma en que éstas se representarán en una base de datos. Esta actividad abarca tanto el modelado lógico y físico de la base de datos (tablas, relaciones, índices, restricciones de integridad, etc.) como el diseño de las estructuras de datos en memoria que utilizarán los componentes y algoritmos del software (listas, árboles, tablas hash, grafos, colas, etc.)[6]. Existe una relación estrecha entre ambos aspectos: las necesidades de información y los

patrones de acceso definidos en la base de datos influyen directamente en las estructuras de datos que se emplean en la aplicación, y a su vez, los requisitos de rendimiento y manipulación de datos de los algoritmos condicionan el diseño del esquema de la base de datos. El SDD [13] refuerza esta visión a través de los puntos de vista de informacion (Information Viewpoint), donde se especifican los elementos de diseño relacionados con la información: las entidades de diseño (elementos de datos, tipos de datos y clases, almacenes de datos y mecanismos de acceso), las relaciones de diseño (asociación, usos, implementaciones), los atributos de datos junto con sus restricciones y relaciones estáticas, así como los atributos de diseño de persistencia y propiedades de calidad.

2.3.5. Diseño de Algoritmos

Esta actividad forma parte del diseño detallado de componentes y se enfoca en la definición de los algoritmos que implementarán la lógica de procesamiento de cada componente. Su objetivo es seleccionar o diseñar soluciones algorítmicas que sean correctas, eficientes y adecuadas a los requisitos de rendimiento, tiempo de respuesta y uso de recursos del sistema. Aunque *Sommerville* [6] la integra dentro del diseño de componentes, estándares como el *IEEE 1016* [13] (a través del punto de vista del algoritmo) y el *SWEBOK* [10] reconocen la importancia de especificar explícitamente los algoritmos.

2.4. Sistemas Multiagentes (MAS)

Un *agente* se define como una entidad computacional autónoma e hilo de ejecución concurrente que encapsula un estado interno, percibe su entorno a través de sensores y actúa sobre él mediante actuadores para alcanzar objetivos específicos dentro de una arquitectura distribuida [14]. Sus características fundamentales abarcan:

- **Situacionalidad (Percepción y Acción):** Capacidad de interactuar de forma directa e inmersa con el entorno, recibiendo estímulos sensoriales y modificando dicho entorno mediante actuadores [14].
- **Autonomía:** Habilidad para tomar decisiones y seleccionar acciones de forma independiente sin intervención externa, preservando su estado interno frente a perturbaciones del sistema [14].
- **Capacidad de Inferencia:** Competencia para procesar información abstracta, deducir observaciones a partir del conocimiento disponible y ejecutar planes orientados

a metas [14].

- **Reactividad y Proactividad:** Habilidad para responder oportunamente ante cambios dinámicos del entorno (reactividad), manteniendo a su vez un comportamiento con iniciativa propia dirigido a metas (proactividad) [14].
- **Comportamiento Social:** Capacidad para comunicarse, cooperar o negociar con otros agentes o humanos mediante el intercambio estructurado de mensajes para resolver problemas complejos [14].

Un *Sistema Multiagentes* (MAS, por sus siglas en inglés) se definen como redes de entidades computacionales autónomas (agentes) que interactúan dentro de un entorno dinámico compartiendo conocimiento y recursos para resolver problemas complejos [14]. Los MAS destacan por su flexibilidad, capacidad de cómputo paralelo, escalabilidad modular y tolerancia a fallos mediante una degradación suave del sistema en caso de avería de alguno de sus componentes [14].

2.4.1. Paradigmas de coordinación y comunicación

La interacción en un *MAS* es indispensable para prevenir conflictos por recursos distribuidos, evitar situaciones de caos y optimizar los objetivos del sistema. La comunicación entre agentes se formaliza mediante *Lenguajes de Comunicación de Agentes* (ACL), como *FIPA-ACL* (Foundation for Intelligent Physical Agents - Agent Communication Language) o *KQML* (Knowledge Query and Manipulation Language), los cuales requieren una ontología compartida y un formato de protocolo estandarizado [14].

La **coordinación** dentro del sistema se articula a través de tres paradigmas principales:

- **Basada en Protocolos:** Emplea mecanismos de negociación distribuida como el *Contract Net Protocol* (CNP), donde los agentes administradores (Managers) subastan contratos a agentes ejecutores (Contractors) mediante licitaciones [14].
- **Basada en Grafos de Coordinación:** Descompone el problema global en subjuegos locales y calcula acciones conjuntas óptimas mediante la combinación lineal de pagos a través de algoritmos como la eliminación de variables [14].
- **Basada en Modelos de Creencias:** Cada agente mantiene un modelo interno del comportamiento de sus vecinos, facilitando la toma de decisiones ágil en entornos dinámicos con restricciones temporales estrictas [14].

2.4.2. Comunicación por pizarra o tablero (Blackboard Communication)

A diferencia del paso directo de mensajes (*message passing*), la comunicación mediante **pizarra o tablero (*Blackboard*)** constituye un esquema centralizado y desacoplado donde un grupo de agentes comparte un repositorio común de información [14].

En este modelo, el *Blackboard* actúa como un almacenamiento compartido para datos de diseño y conocimiento de control. El acceso a esta memoria compartida se gestiona mediante una interfaz de red o *control shell*, encargada de notificar a los agentes cuando existe información relevante disponible. Aunque esta arquitectura simplifica la coordinación de grupos de agentes remotos, su principal limitación radica en que la pizarra representa un punto único de fallo (*single point of failure*), por lo que una avería central puede inhabilitar al grupo completo [14].

2.5. Modelos grandes de lenguaje (LLM)

Los modelos grandes de lenguaje (LLM, por sus siglas en inglés) son arquitecturas avanzadas basadas principalmente en la red neuronal *Transformer* que procesan y generan lenguaje natural mediante un entrenamiento masivo sobre extensos corpus de texto [15], [16], [17]. A través de esta etapa de pre-entrenamiento, el modelo comprime conocimiento lingüístico y conceptual de amplio dominio, lo que le permite identificar patrones semánticos y estructurales complejos de manera implícita. Como resultado, los LLM actúan como herramientas de propósito general capaces de resolver directamente tareas de redacción, análisis de texto, traducción y síntesis de información sin requerir entrenamiento específico, si no, pueden usar una amplia diversidad de fuentes [15], [16], [17].

2.5.1. Principios de inferencia y generación de texto en modelos de lenguaje

La inferencia en los LLM se fundamenta en un proceso autorregresivo de predicción del siguiente token, condicionando probabilísticamente cada palabra o subpalabra generada al contexto previo y al *prompt* de entrada [16]. Esta arquitectura habilita la adaptación del comportamiento del modelo mediante el aprendizaje en contexto (*In-Context Learning* o ICL) y el seguimiento de instrucciones, respondiendo conforme a las restricciones y formatos especificados [17]. Sin embargo, desde una perspectiva operacional, los LLM se

comportan como sistemas de "caja negra", donde la precisión y el estilo de la respuesta varían sustancialmente entre modelos en función de la arquitectura de la red, la escala y diversidad de los datos de entrenamiento, la ventana de contexto y la calidad del *prompt* proporcionado [16], [17], [18].

2.5.2. Capacidades de los LLMs en la comprensión, diseño y generación de código

En la ingeniería de software y el desarrollo de sistemas, las capacidades de síntesis y comprensión de código de los LLM se derivan de la inclusión de vastos repositorios de software y documentación técnica durante su pre-entrenamiento [16]. La exposición a esta información especializada capacita al modelo para reconocer dependencias de largo alcance, sintaxis estructurada y patrones lógicos de programación [17]. Durante la fase de diseño de software, el modelo puede evaluar las especificaciones proporcionadas en el contexto y recomendar patrones de diseño adecuados, por ejemplo evaluar la arquitectura de un sistema y sugerir un enfoque Modelo-Vista-Controlador (MVC) al contrastar los requerimientos recibidos con las abstracciones aprendidas en su entrenamiento [15], [16]. Asimismo, los LLM aceleran el ciclo de vida del desarrollo mediante la generación automática de código, la refactorización, el depurado y la creación de scripts de prueba automatizados [15], [16].

2.5.3. Limitaciones actuales: Alucinaciones, ventanas de contexto y deficiencias en el razonamiento algorítmico determinista

A pesar de sus amplias capacidades, los LLM presentan importantes limitaciones técnicas e inherentes [16], [18]. En primer lugar, el fenómeno de la *alucinación* ocurre cuando el modelo genera información inconsistente o no verificable respecto a las fuentes factuales existentes, debido a imprecisiones en los datos de entrenamiento, sesgos en las estrategias de muestreo estocástico y la naturaleza probabilística del objetivo de predicción de texto [15], [16]. En segundo lugar, las restricciones en la ventana de contexto condicionan el volumen de información procesable simultáneamente; aunque el texto se segmenta en tokens (unidades discretas de representación), secuencias de entrada extensas provocan degradación en la atención sobre detalles intermedios, fenómeno conocido como *lost in the middle* [17]. Por último, existen marcadas deficiencias en el razonamiento algorítmico determinista y la ejecución simbólica [18]. Incluso los modelos de razonamiento avanzado

(*Large Reasoning Models* o LRM) diseñados para generar cadenas de pensamiento (*Chain-of-Thought*), sufren una caída drástica de precisión hasta el colapso total al incrementar la complejidad del problema [18]. Esto demuestra que los modelos no ejecutan algoritmos explícitos de manera rigurosa ni se benefician de la inclusión del algoritmo en el prompt, sino que dependen de la imitación probabilística de patrones aprendidos [18].

2.6. Árbol de búsqueda de Monte-Carlo (MCTS)

Un *Monte Carlo Tree Search*, (MCTS) es un algoritmo de búsqueda y toma de decisiones que construye un árbol de forma incremental mediante simulaciones estocásticas para evaluar opciones y elegir la acción más prometedora, cada nodo es un estado o configuraciones de un problema y las aristas son transiciones (acciones) de nodo a nodo [19].

El ciclo iterativo consta de cuatro fases: *selección, expansión, simulación y propagación hacia atrás* (retropropagacion).

- **Selección**: Se navega por el árbol desde la raíz hasta un nodo hoja, eligiendo nodos "buenos", estos nodos se etiquetan así al equilibrar dinámicamente dos objetivos: la *explotación* (favorecer estados con un historial de alto rendimiento o recompensa acumulada) y la *exploración* (visitar estados poco explorados para reducir la incertidumbre y evitar mínimos locales) [19].

- **Expansión**: A partir del nodo hoja seleccionado, se añade uno o más nodos hijos para incorporar nuevas acciones (permitidas y viables) del espacio de búsqueda que aún no han sido exploradas en el árbol [19].

- **Simulación**: Se realiza un recorrido (frecuentemente aleatorio o guiado por heurísticas) desde el nodo recién expandido hasta alcanzar un estado terminal, lo que permite obtener una estimación empírica del valor del estado [19].

- **Retropropagacion**: El resultado de la simulación se propaga de forma ascendente desde el nodo evaluado hasta la raíz, actualizando las estadísticas de todos los nodos visitados en esa iteración (incrementando su contador de visitas y ajustando su valor acumulado o promedio) [19].

2.7. Ontología como representación de conocimiento

Una ontología es una especificación formal y explícita de una conceptualización compartida de un dominio [20], [21]. En el ámbito de las ciencias de la computación y la representación del conocimiento, las ontologías proporcionan un vocabulario estructurado que hace explícito el significado de los datos y las relaciones entre entidades [22]. Cumplen un doble propósito: un propósito **representacional**, enfocado en la enumeración declarativa de los hechos del dominio, y un propósito **computacional**, orientado a la derivación de inferencias lógicas a partir de los hechos representados. Esta capacidad inferencial permite que sistemas de software e inteligencia artificial procesen, compartan, reutilicen y comprendan datos de forma autónoma, sin requerir intervención humana previa ni conocimientos precodificados.

2.7.1. Definición y estructura de las ontologías computacionales (clases, atributos, relaciones)

Una ontología computacional organiza el conocimiento de un dominio a través de varios componentes lógicos interrelacionados:

- **Clases o Conceptos**: Representan las categorías generales o conjuntos de objetos dentro del dominio de discurso. Se estructuran mediante jerarquías de subclases, donde los conceptos más específicos heredan las propiedades y características de las categorías más generales [20].
- **Relaciones o Propiedades de Objeto**: Definen los vínculos y asociaciones entre diferentes clases o individuos. Permiten especificar cómo interactúan las entidades e incluyen restricciones de dominio (qué conceptos pueden originar la relación) y de rango (qué conceptos pueden recibirla), además de características lógicas como transitividad, simetría, relaciones inversas, entre otras. [22].
- **Atributos o Propiedades de Datos**: Asocian a los individuos o instancias con valores de datos concretos y literales, tales como números enteros, cadenas de texto, fechas o valores booleanos.
- **Individuos o Instancias**: Corresponden a los objetos o entidades concretos y particulares que existen dentro del dominio y que pertenecen a una o más clases específicas [20].

- **Axiomas y Restricciones**: Reglas y afirmaciones lógicas que definen las condiciones necesarias y suficientes para la pertenencia a un concepto, restringen la interpretación del vocabulario y garantizan la consistencia lógica del sistema para fundamentar el razonamiento automático [20], [22].

2.7.2. Lenguajes de modelado ontológico

Los lenguajes de modelado ontológico han evolucionado con el objetivo de equilibrar la riqueza expresiva en la representación del conocimiento con la eficiencia computacional durante el proceso de inferencia automática [22]. Entre las tecnologías y estándares principales destacan:

- **RDF y RDFS**: El Marco de Descripción de Recursos (*Resource Description Framework*, RDF) proporciona una estructura básica basada en triples formados por sujeto, predicado y objeto para declarar datos en la Web. Por su parte, *RDF Schema* (RDFS) extiende este marco ofreciendo un vocabulario para organizar clases en jerarquías y definir el dominio y el rango de las propiedades [22].

- **Lógicas de Descripción**: Constituyen una familia de formalismos lógicos derivados de la lógica de primer orden que proveen una semántica formal rigurosa basada en teoría de modelos, garantizando procedimientos de decisión decidibles para la deducción automática [22].

- **OWL (Web Ontology Language)**: Es el estándar del W3C diseñado para la representación de ontologías en la Web Semántica, construido sobre RDF/RDFS y fundamentado en las Lógicas de Descripción [22]. OWL amplía la capacidad expresiva de RDFS al permitir combinaciones lógicas de clases (unión, intersección y complemento), la declaración de clases disjuntas, restricciones locales sobre propiedades y restricciones de cardinalidad. OWL se divide en tres sublenguajes o perfiles adaptados a distintos requerimientos [22]:

  1. **OWL Lite**: Ofrece una sintaxis reducida y menor complejidad computacional, orientada a usuarios que requieren jerarquías simples y restricciones básicas con un razonamiento rápido.
  2. **OWL DL**: Basado en lógicas de descripción expresivas, garantiza la decidibilidad del razonamiento (asegurando que los algoritmos de inferencia siempre

terminan en un tiempo finito) mientras mantiene una elevada capacidad de representación formal.

3. **OWL Full**: Ofrece máxima compatibilidad semántica con RDF y permite el metamodelado (como tratar clases como si fuesen individuos), aunque pierde la garantía de decidibilidad computacional.

2.7.3. Modelado semántico para la taxonomía y clasificación de problemas algorítmicos

El modelado semántico resulta fundamental para estructurar la caracterización de problemas de decisión y guiar la selección adaptativa de algoritmos en entornos computacionales dinámicos [23]. El trabajo de Duran Et. Al. [24] introduce la taxonomía **TDMIoT** (*Taxonomy for Decision-Making in IoT Systems*), la cual conceptualiza el proceso de decisión como una asociación estructurada entre un problema de decisión y una solución algorítmica. TDMIoT organiza el dominio a través de siete dimensiones clave organizadas en dos espacios:

- **Espacio del Problema**: Se define mediante el *Objetivo* principal (pronóstico, optimización, planificación, resolución de conflictos o predicción esta última subclasificada en estimación, regresión y clasificación), el tipo de *Procesamiento* (centralizado o descentralizado), la *Capa de procesamiento* (percepción, middleware o aplicación), el tipo de *Interacción* (del sistema o humana) y la fuente de *Incertidumbre* (de medición, de parámetros o de modelo).

- **Espacio de la Solución**: Se caracteriza mediante la *Representación del conocimiento* (basada en datos, simbólica, matemática o híbrida) y el *Enfoque algorítmico* (guiado por criterios, guiado por datos, guiado por utilidad o guiado por probabilidad).

Sobre la base conceptual de esta taxonomía, [23] propone la ontología **O-DMIoT** (*Ontology for Decision-Making in the IoT*), fundamentada en la ontología superior UFO (*Unified Foundational Ontology*) y desarrollada mediante la metodología de ingeniería ontológica SABiO. O-DMIoT formaliza el conocimiento organizándolo en capas lógicas interconectadas:

- **Capa de Requerimientos**: Caracteriza los problemas de decisión e independiza la definición del problema de la estrategia algorítmica, formalizando especificaciones

de entrada, salida, parámetros y criterios de rendimiento.

- **Capa de Diseño**: Especifica las estrategias algorítmicas, supuestos, complejidad y restricciones teóricas del algoritmo sin vincularse al entorno de ejecución.
- **Capa de Ejecución**: Operativiza el algoritmo instanciándolo como un servicio desplegado dentro de la infraestructura tecnológica.
- **Rol de Evaluación**: Actúa de manera transversal asociando métricas de Calidad de Servicio (QoS), Calidad de Experiencia (QoE) y características intrínsecas del algoritmo (complejidad temporal y espacial) para evaluar el desempeño.

Este marco semántico permite abordar la selección de algoritmos como un proceso de planificación secuencial guiado por razonamiento ontológico y aprendizaje automático, posibilitando la reconfiguración adaptativa de servicios en tiempo de ejecución [23].

Capítulo 3

Estado del Arte

Este capítulo examina el estado del arte relacionado con el diseño automatizado de software y el concepto de *Self-Design*. Dado el carácter emergente de esta línea de investigación, la literatura disponible es aún limitada y se encuentra distribuida en áreas afines con diversos enfoques. Por ello, este capítulo adopta presentar resultados relevante obtenidos mediante una Revisión Sistemática de la Literatura descrita en el Capítulo 5. Se realizo con un enfoque exploratorio orientado a identificar conceptos, tendencias, enfoques y mecanismos relevantes que permitan evaluar trabajos relacionados con el problema y área de dominio de esta misma investigación.

3.1. Conceptualización del Auto-Diseño

En la vasta literatura sobre sistemas auto-adaptativos y auto-gestionados con origen en [25], [26], [27] se encuentran diversos enfoques para cubrir parcialmente la necesidad de intervención humana. Lo que se considera auto-diseño busca la completa independencia del sistema en ese sentido. Sin lugar a dudas, el trabajo de Porter Et. Al. [28] representa uno de los acercamientos más directos al concepto. Los autores conciben el *Self-Designing Software* como un paradigma en el cual el propio software se convierte en un miembro activo de su equipo de desarrollo. Su técnica principal radica en el *Hot-Swapping* o intercambio en caliente, el cual permite sustituir componentes de software en un sistema en ejecución de manera segura mediante el uso de bloques de construcción (*building blocks*) validados y provistos de interfaces bien definidas.

Sin embargo, desde la perspectiva de la ingeniería de software clásica, este enfoque no satisface completamente el ciclo de diseño, ya que no concibe el diseño como el conjunto

de actividades estructuradas que entregan artefactos de diseño formales (como diagramas arquitectónicos, especificaciones de componentes, interfaces, etc.) [7], si no un reemplazo de componentes establecidos dependiendo de la funcionalidad.

Con una especificación mas concreta centrada en estructuras de datos, el trabajo Idreos Et. Al. [29] describe una metodología para automatizar el diseño de estructuras de almacenamiento personalizadas. En lugar de depender de diseños tradicionales e inflexibles, este enfoque construye estructuras a la medida partiendo de un conjunto de primitivas de diseño lógicas y físicas clasificadas en su "Tabla Periódica de las Estructuras de Datos"[30]. El conjunto combinatorio de estas primitivas conforma un vasto espacio de diseño en el cual se evalúan rigurosamente los compromisos entre rendimiento y utilización de recursos. Tras este análisis, el candidato óptimo es enviado a un generador de código que sintetiza el diseño en código C++ ejecutable para el sistema final.

De manera complementaria en esta especificación, la herramienta COZY desarrollada por Loncaric et al. [31], [32] presenta un marco de síntesis lógica de colecciones de datos. La entrada del sistema consiste en una especificación declarativa de alto nivel que describe el estado abstracto (datos lógicos como usuarios, grupos o membresías), las consultas requeridas y las operaciones de actualización. A través de un bucle iterativo de refinamiento, incrementalización y verificación formal apoyado por el resolvedor SMT Z3 (mediante síntesis inductiva guiada por contra ejemplos, CEGIS), COZY genera estructuras de datos intrusivas y seguras por diseño, listas para su compilación.

3.2. Automatización del Diseño

La automatización de las tareas de diseño se ha abordado desde múltiples paradigmas de cómputo, respondiendo a la complejidad del dominio y al nivel de abstracción objetivo.

3.2.1. Auto-diseño basado en Inteligencia Artificial y Aprendizaje

En la generación de lógica y algoritmos, [33] proponen el marco ALDes (*Autoregressive Learning-based Designer*), el cual formula el diseño de algoritmos metaheurísticos como una tarea de generación de secuencias. A diferencia de los métodos tradicionales con estructuras fijas, ALDes utiliza redes autorregresivas basadas en *Transformers* para inferir secuencias con estructuras de control arbitrarias (bucles, condicionales) y longitudes variables, permitiendo la síntesis autónoma de algoritmos adaptados a problemas específicos. Por su parte, Yi y Qu [34] exploran el uso de Aprendizaje por Refuerzo (*Proximal Polici*

Optimisation - PPO) sobre un espacio de características dependientes de la instancia y del estado de búsqueda, logrando la combinación dinámica de componentes algorítmicos.

3.2.2. Autodiseño bio-inspirado, de reúso y búsqueda

Dentro de la *Ingeniería de Software Basada en Búsqueda* (SBSE), Ramírez et al. [35] presentan un enfoque evolutivo para el descubrimiento e inferencia autónoma de arquitecturas de software a partir de diagramas de clases. La aproximación utiliza algoritmos genéticos guiados por métricas de cohesión y acoplamiento para agrupar clases en componentes y descubrir interfaces arquitectónicas. En el ámbito de la refactorización estructural, Houichime y El Amrani [36] proponen el marco ODR, el cual codifica las propiedades del software mediante números complejos en un espacio vectorial unificado, permitiendo aplicar heurísticas de optimización para refactorizar y mejorar la calidad del diseño de forma directa.

3.2.3. Autodiseño basado en modelado de agentes

La emergencia de los Grandes Modelos de Lenguaje (LLMs) ha impulsado el desarrollo de sistemas multi-agente para la ingeniería de requerimientos y el diseño arquitectónico. Dam et al. [37] introducen un marco conceptual multi-agente donde agentes autónomos interpretan requerimientos en lenguaje natural, generan modelos UML y colaboran en la resolución de conflictos semánticos y fusión de versiones.

Asimismo, Zhang et al. [38] proponen el marco MAAD (*Multi-Agent Architecture Design*), en el cual agentes especializados (Analista, Modelador, Diseñador y Evaluador) simulan el flujo de trabajo de un equipo humano para traducir especificaciones SRS en planos arquitectónicos verificados sin incurrir en alucinaciones típicas de esquemas *zero-shot*. De igual forma, Rajendran et al. [39] diseñan un entorno multi-agente colaborativo y competitivo que emplea protocolos de subasta y consenso para balancear atributos de calidad en conflicto (rendimiento, seguridad y mantenibilidad) durante la refactorización arquitectónica.

3.3. Conceptos Emergentes y Mecanismos de Control en Autodiseño

El auto-diseño requiere motores de ejecución capaces de efectuar cambios en tiempo de ejecución de manera segura y controlada. Affonso et al. [40] presentan la arquitectura de referencia RA4SaS, basada en reflexión computacional, la cual permite inspeccionar, desensamblar y reconstruir entidades orientadas a objetos en tiempo de ejecución mediante metadatos y plantillas.

Por otro lado, Weyns y Andersson [41] formalizan la frontera entre la auto-adaptación convencional y la auto-evolución basándose en el concepto de Dominio de Diseño Operacional (ODD). Los autores demuestran que cuando un sistema enfrenta requerimientos o entornos no previstos en su ODD inicial ($ODD_{Se} = ODD_{Si} \cup ODD_e$), la simple adaptación de parámetros es insuficiente, obligando al sistema a evolucionar su propia estructura de manera autónoma.

3.4. Comparación de Trabajos

A continuación, la Tabla 3.1 sintetiza y clasifica las contribuciones clave revisadas en la literatura según su nivel de abstracción, técnica empleada, fase de aplicación y el artefacto generado.

Tabla 3.1: Matriz comparativa de enfoques de automatización y artefacto de software.

Autor	**Nivel de Abstracción**	**Técnica Principal**	**Artefacto Generado**
Porter et al. [28]	Componentes / Módulos	Hot-Swapping & Meta-composers	Ensamblado dinámico
Idreos et al. [29]	Estructuras de Datos	Primitivas de Layout & Cost Models	Código C++ optimizado
Loncaric et al. [31]	Estructuras de Datos	Síntesis Lógica & CEGIS (Z3)	Estructuras intrusivas C++
Zhao et al. [33]	Algoritmos / Heurísticas	Transformers Autorregresivos	Algoritmos de búsqueda
Ramírez et al. [35]	Arquitectura de Software	Algoritmos Evolutivos (SBSE)	Componentes e Interfaces
Zhang et al. [38]	Arquitectura de Software	LLMs Multi-Agente & Consenso	Diagramas UML / SRS
Rajendran et al. [39]	Arquitectura / Refactorización	Agentes Especializados & Subastas / Runtime	Código Refactorizado
Affonso et al. [40]	Entidades de Software	Reflexión Computacional	Reconfiguración estructural
Weyns et al. [41]	Marco de Auto-Evolución	Análisis de ODD	Extensión del Dominio *ODD*

Capítulo 4

Estado del Arte y Actividades de Auto-Diseño

Este capítulo examina el estado del arte en el diseño automatizado de software y el paradigma de *Self-Design*. Para organizar los hallazgos de la Revisión Sistemática de la Literatura descrita en el , este capítulo analiza las contribuciones según las actividades de diseño de software que automatizan (diseño arquitectónico, diseño de componentes, estructuras de datos y algoritmos), complementadas por una dimensión transversal de soporte en tiempo de ejecución (*runtime*). Esta perspectiva permite evaluar críticamente los artefactos producidos en cada etapa y evidenciar el vacío de investigación existente hacia un auto-diseño de software integral.

4.1. Automatización de las Actividades de Diseño de Software

El proceso de ingeniería de software involucra actividades de diseño a diversos niveles de abstracción. La literatura actual aborda estas actividades de forma especializada:

4.1.1. Actividades de Diseño Arquitectónico (Macro-nivel)

Las actividades de macro-diseño abordan la traducción de especificaciones de requerimientos en representaciones arquitectónicas de alto nivel. La emergencia de los Grandes Modelos de Lenguaje (LLMs) y los sistemas multi-agente ha impulsado de forma decisiva esta área. Dam et al. [37] introducen un marco conceptual donde agentes autónomos

interpretan requerimientos en lenguaje natural, generan modelos UML y colaboran en la resolución de conflictos semánticos. En este ámbito, el trabajo de Zhang et al. [38] con el marco MAAD (*Multi-Agent Architecture Design*) representa el antecedente más cercano a la automatización del diseño arquitectónico. MAAD orquesta cuatro agentes especializados (Analista, Modelador, Diseñador y Evaluador) para traducir documentos de requerimientos (SRS) en modelos de vistas "4+1"de Kruchten, diagramas UML y reportes de evaluación ATAM. Por su parte, Rajendran et al. [39] emplean protocolos de subasta entre agentes para balancear atributos de calidad en conflicto durante la reestructuración arquitectónica. A pesar de sus aportes en el modelado conceptual, las herramientas de este nivel operan exclusivamente en fase estática (*design-time*) como asistentes documentales, careciendo de integración con motores de ejecución autónoma o de la capacidad de generar código o evolucionar dinámicamente en tiempo de ejecución.

4.1.2. Actividades de Diseño de Componentes e Interfaces (Meso-nivel)

En el meso-nivel, la literatura se concentra en la identificación de límites modulares, la inferencia de interfaces y la refactorización estructural. Dentro de la Ingeniería de Software Basada en Búsqueda (SBSE), Ramírez et al. [35] aplican algoritmos genéticos para agrupar diagramas de clases en componentes e inferir sus interfaces. En la refactorización estructural, Houichime y El Amrani [36] proponen el marco ODR, el cual codifica las propiedades orientadas a objetos mediante números complejos para aplicar heurísticas de optimización vectorial. Por su parte, Porter et al. [28] conciben el *Self-Designing Software* mediante el intercambio en caliente (*Hot-Swapping*) de bloques de construcción (*building blocks*) predefinidos. No obstante, desde la perspectiva de la ingeniería de software clásica, este enfoque automatiza la sustitución de módulos en *runtime*, pero asume que los componentes, sus interfaces y su arquitectura ya han sido concebidos y validados previamente por diseñadores humanos.

4.1.3. Actividades de Diseño de Estructuras de Datos (Micro-nivel de Almacenamiento)

En las actividades de micro-diseño orientadas al almacenamiento, la automatización busca reemplazar la selección manual de estructuras de datos por síntesis formal y modelos de costo. Idreos et al. [29], [30] desarrollan la "Tabla Periódica de las Estructuras

de Datosz el *Data Alchemist*, un marco que sintetiza estructuras de almacenamiento personalizadas en código C++ ejecutable combinando primitivas físicas y lógicas. De forma complementaria, Loncaric et al. [31], [32] presentan COZY, una herramienta de síntesis lógica apalancada en el resolvedor SMT Z3 y bucles CEGIS para generar colecciones de datos intrusivas a partir de especificaciones declarativas.

4.1.4. Actividades de Diseño de Algoritmos y Heurísticas (Micro-nivel Lógico)

Las actividades de micro-diseño lógico se enfocan en la generación autónoma de la lógica procedimental y estrategias de resolución de problemas. Zhao et al. [33] proponen ALDes, formulando el diseño de algoritmos metaheurísticos como una tarea autorregresiva secuencia a secuencia basada en *Transformers*, lo que permite inferir estructuras de control complejas (bucles, condicionales). Asimismo, Yi y Qu [34] exploran el uso de Aprendizaje por Refuerzo (PPO) para la combinación dinámica de componentes algorítmicos.

4.2. Dimensión Transversal: Soporte de Ejecución y Auto-Evolución

El auto-diseño de software requiere motores capaces de efectuar cambios en tiempo de ejecución de manera segura y controlada. Affonso et al. [40] presentan la arquitectura de referencia RA4SaS, basada en reflexión computacional, la cual permite inspeccionar, desensamblar y reconstruir entidades orientadas a objetos en *runtime* mediante metadatos y plantillas. A nivel teórico, Weyns y Andersson [41] formalizan la frontera entre la auto-adaptación y la auto-evolución basándose en el Dominio de Diseño Operacional (ODD). Demuestran que cuando un sistema enfrenta requerimientos o entornos no previstos en su ODD inicial ($ODD_Se = ODD_Si \cup ODD_e$), la simple adaptación de parámetros resulta insuficiente, obligando al sistema a evolucionar su propia estructura autónomamente.

4.3. Síntesis Comparativa y Declaración del Vacío de Investigación

La Tabla clasifica y evalúa de forma sistemática los trabajos revisados según la actividad de diseño que realizan, sus omisiones estructurales y su brecha frente al auto-diseño completo.

Tabla 4.1: Matriz comparativa de enfoques de automatización según actividades de diseño y brecha de Self-Design.

Enfoque / Autor	**Actividad de Diseño**	**¿Qué generan? (Artefacto)**	**¿Qué NO generan? (Omisión)**	**Brecha frente a *Self-Design* Completo**
Zhang et al. (MAAD) [38]	Diseño Arquitectónico (Macro)	Modelos de vistas "4+1", UML, SRS y reportes ATAM.	Código ejecutable y componentes operacionales en runtime.	Operación estática en *design-time*; actúa como asistente documental sin acoplamiento a ejecución.
Dam et al. [37]	Diseño Arquitectónico (Macro)	Modelos UML conceptuales y resolución de conflictos.	Código fuente e infraestructura de ejecución.	Enfoque conceptual desacoplado de la dinámica operativa y del ciclo adaptativo.
Ramírez et al. [35]	Componentes e Interfaces (Meso)	Agrupamiento modular e inferencia de interfaces.	Arquitectura *ex nihilo*, lógica interna y código ejecutable.	Requiere diagramas de clases preexistentes; refactoriza pero no diseña desde cero.
Houichime et al. (ODR) [36]	Componentes e Interfaces (Meso)	Refactorización estructural por vectorización compleja.	Diseños arquitectónicos nuevos o lógica interna.	Depende estrictamente de código base existente para optimizar métricas.
Porter et al. [28]	Componentes e Interfaces (Meso)	Reconfiguración dinámicas por *Hot-Swapping*.	Artefactos formales de diseño (modelos UML) y código nuevo.	Sustituye bloques preexistentes hechos por humanos; no realiza la actividad de diseño.
Idreos et al. [29]	Estructuras de Datos (Micro)	Código C++ optimizado para almacenamiento.	Arquitectura de software, componentes o modelos de dominio.	Hiper-especializado en almacenamiento a micro-nivel; ignora el contexto global.
Loncaric et al. (Cozy) [31]	Estructuras de Datos (Micro)	Colecciones C++ intrusivas por síntesis lógica (CEGIS/Z3).	Módulos de software, interfaces o diagramas arquitectónicos.	Limitado a la síntesis de colecciones aisladas; carece de visión arquitectónica.
Zhao et al. (ALDes) [33]	Algoritmos y Heurísticas (Micro)	Algoritmos metaheurísticos por *Transformers*.	Estructuras de datos complejas, módulos o arquitectura.	Generación acotada a secuencias lógicas de optimización; no razona sobre arquitectura.
Affonso et al. [40]	Soporte de Ejecución (Transversal)	Reconfiguración estructural en runtime por reflexión.	Artefactos de diseño conceptual o síntesis arquitectónica.	Infraestructura de soporte en runtime; requiere plantillas y modelos preconcebidos.
Weyns et al. [41]	Auto-Evolución (Transversal)	Marco teórico para extensión del dominio *ODD*.	Artefactos formales de diseño o código de software concreto.	Formaliza las fronteras de auto-evolución, pero carece de motor de síntesis de diseño.

4.3.1. Declaración del Vacío de Investigación (Research Gap)

El análisis comparativo de la literatura evidencia un vacío fundamental: ninguno de los trabajos revisados aborda de manera simultánea la generación autónoma de artefactos formales de diseño de software articulada con un entorno de ejecución e integración continua en tiempo de ejecución. Esta limitación general se sintetiza en tres brechas principales:

1. Especialización Fragmentada por Actividad: Las soluciones orientadas a actividades de micro-diseño (Idreos, Loncaric, Zhao) logran precisión en la síntesis de código de datos o algoritmos, pero desconocen la estructura arquitectónica global. Inversamente, los enfoques de macro-diseño (MAAD, Dam) generan abstracciones del sistema pero no descienden a la síntesis de código o componentes operacionales.

2. El Límite de MAAD: Si bien MAAD [38] representa la propuesta más avanzada en la automatización del diseño arquitectónico mediante sistemas multi-agente, su alcance se detiene en la producción de documentación estática en *design-time*, sin conexión con motores de ejecución, verificación o evolución dinámica en *runtime*.

3. Desconexión entre Diseño y Mecanismos de Ejecución: Los marcos de auto-adaptación y auto-evolución (Porter, Affonso, Weyns) proveen los motores de reflexión y formalismos (ODD) para modificar el software en tiempo de ejecución, pero presuponen que los componentes y modelos de diseño ya han sido creados previamente por ingenieros humanos.

Por lo tanto, se justifica la necesidad de una propuesta de investigación que unifique la generación autónoma de artefactos formales de diseño con un motor de ejecución y auto-evolución continua ante entornos inciertos.

Capítulo 5

Propuesta

5.1. Metodología

La metodología consta de dos etapas principales, la primera que fue realizar una revisión sistemática de literatura sobre técnicas, marcos de trabajo y demás propuestas que llevaran a cabo el autodiseño, esta misma se llevo a cabo siguiendo la metodología prisma, los resultados obtenido ayudaron a fundamentar la toma de decision para construir la propuesta. La siguiente etapa fue la construcción de un sistema multiagentes, se compararon diferentes metodologías, pero se decanto por Prometheus ya que implementa los conceptos BDI de manera natural, la comparación con metodologías esta dada con (Gaia, Tropos, Mase, O-mase, Adelfe, Pass)

5.1.1. Revisión Sistemática de literatura

El primer paso para desarrollar y fundamentar mi propuesta fue realizar un revisión sistemática de literatura, en la que siguiendo los lineamientos PRISMA [42], con este proceso estructurado se realizo la revisión de manera exhaustiva, el objetivo principal de esta Systematic Literature Review (SLR), es conocer como se ha automatizado el diseño de sistemas, conocer el estado del arte permite reconocer vacíos en el mismo.

5.1.1.1. Formulación preguntas de investigación

La formulación de una pregunta o preguntas de investigación en cualquier investigación es la piedra angular de la misma, en una SLR no es la excepción, para esto se utilizo el marco de trabajo PICO [43], La Población (P), abarca los procesos de diseño de software,

desde las tareas tradicionales automatizadas, hasta el auto-diseño bajo el marco de los sistemas de auto-ingeniería (SES) o que tengan relación directa con ellos. La intervención (I) se refiere a todos los enfoques automatizados, autónomos o impulsados por IA dentro de la basta literatura, para apoyar a dichas actividades. Estos enfoques comprenden modelos, técnicas y mecanismos destinados a permitir que los sistemas generen, evalúen, o evoluciones artefactos de diseño de manera autónoma. La Comparación (C), implica un análisis comparativo entre las metodologías o técnicas convencionales, contra enfoque progresivamente automatizados. Finalmente, el resultado previsto, denominado Resultado (Outcome en ingles), destaca en una síntesis sistemática del conocimiento existente para mapear esta evolución tecnológica, evaluar las teorías e identificar las brechas de investigación pendientes que obstaculizan la realización de un verdadero auto-diseño.

Con base a esta formulación las siguientes preguntas son propuestas:

- **RQ1:** ¿De que maneras se han automatizado o hecho autónomos los procesos tradicionales de ingeniería de diseño de software?
- **RQ2:** ¿Que términos o conceptos emergentes son equivalentes o cercanos a una actividad de diseño totalmente autónoma?
- **RQ3:** ¿Como se ha abordado el **Autodiseño** de sistemas en el contexto de los SES?

5.1.1.2. Estrategia de búsqueda

Para contestar a la primer pregunta de investigación (RQ1), el análisis fue estructurado alrededor de las actividades en el diseño de software, considerando su correspondencia con enfoques automáticos o autonómico, las etapas se muestran en la tabla 5.1.

Tabla 5.1: Actividades de diseño y sus correspondientes artefactos de diseño.

Actividades de diseño	**Artefactos de diseño**
Diseño de Arquitectura	Arquitectura del sistema
Diseño de interfaz	Especificación de interfaces
Diseño de componentes	Especificación de componentes
Diseño de estructuras de datos	Especificación de estructuras de datos
Diseño de algoritmos	Especificación de algoritmos

Esta descomposición facilita evaluar en que medida cada fase del diseño ha sido reinterpretada, respaldada o automatizada bajo los principios de automatización de auto-gestión.

Esto ofrece una visión amplia del diseño en nuestra búsqueda. De tal manera que una búsqueda granular se consigue con sub-preguntas de investigación, al final se contesta la RQ1.

- **SRQ1.1** ¿De qué manera se ha conceptualizado, implementado o evaluado el autodiseño arquitectónico o la automatización del diseño arquitectónico en la literatura?
- **SRQ1.2** ¿Cómo se ha generado o abordado la noción de autodiseño de interfaces o de diseño automatizado de interfaces en la investigación existente?
- **SRQ1.3** ¿En qué medida se ha explorado y tratado el autodiseño de componentes o el diseño automatizado de componentes en estudios previos?
- **SRQ1.4** ¿Cómo se han definido e investigado en la literatura los enfoques sobre el autodiseño de estructuras de datos o el diseño automatizado de estructuras de datos?
- **SRQ1.5** ¿Cómo se ha abordado el autodiseño de algoritmos o el diseño automatizado de algoritmos?

Para proveer de una respuesta mas comprensiva a la segunda pregunta de investigación (RQ2), se realizo una búsqueda exploratoria, usando terminología emergente. Este paso apunta a identificar conceptos e investigaciones que de una manera no explicita busca el auto-diseño. como resultado tenemos tópicos como diseño evolucionario de sistemas de software, diseño generativo en software, automatización del diseño de software e inteligencia artificial para ingeniería de software (AI4SE), estos tópicos se añadieron a la estrategia de búsqueda. Por ultimo, la manera de abordar la tercer pregunta de investigación (RQ3), se adopto una estrategia dirigida por el concepto clave de auto-diseño, algo que de manera explicita nos dirigiera a esto. así que se suman los siguiente términos: Auto-diseño, Sistemas de Auto-Arquitectura y Sistemas autosuficientes.

Las búsquedas de la SLR fue realizada en cuatro bases de datos de alto impacto en el campo de la ingeniería de software: *Scopus, Web of Science, IEEE Xplore y ACM Digital Library*. Estas plataformas se seleccionaron debido a la disponibilidad de acceso institucional completo y a su amplia cobertura de publicaciones revisadas por pares, incluidos artículos de revistas, actas de conferencias y capítulos de libros relevantes para la inteligencia artificial, la ingeniería de software y los sistemas autónomos y autonómicos. *Google Scholar* si bien es un buen motor de búsqueda, no es una base de datos de manera directa y presenta limitaciones en la replicabilidad de las búsquedas, por lo tanto no fue una fuente primaria de búsqueda [44], sin embargo en el proceso de bola de nieve (snowballing) si se uso para la búsqueda de las citas, tanto de entrada como de salida.

Las estrategias de búsqueda se elaboraron sistemáticamente combinando términos relacionados con el tema mediante operadores booleanos (`AND`, `OR`) para estructurar explícitamente sus relaciones lógicas. La búsqueda se restringió al campo `TITLE-ABS-KEY` con el fin de aumentar la precisión conceptual y reducir el ruido en la recuperación de información. Posteriormente, se aplicaron criterios de inclusión y exclusión predefinidos para refinar y validar el conjunto de estudios resultante. Un ejemplo de una consulta completa esta escrita en la tabla 5.2.

Tabla 5.2: Ejemplo de una consulta de búsqueda completa (Scopus)

Database	Scopus
Search Field	TITLE-ABS-KEY
Time Range	2015–2026
Subject Areas	Computer Science (COMP), Engineering (ENGI)
Complete Query	`TITLE-ABS-KEY ((("self-architecting" OR "self architectural" OR " auto-architecting" OR "auto architectural") W/3 "system*") OR (("self-architectural" OR "auto-architectural") W/3 "design")) AND PUBYEAR > 2014 AND PUBYEAR < 2027 AND (LIMIT-TO (SUBJAREA, "COMP") OR LIMIT-TO (SUBJAREA, "ENGI"))`

Las consultas presentadas a continuación corresponden a un conjunto resumido de expresiones que inicialmente fueron ejecutadas en Scopus, y posteriormente adaptadas a las sintaxis especificas de las diferentes bases de datos, se listan las consultas en la tabla 5.3.

Tabla 5.3: Resumen de las consultas en Scopus (Obviando los filtros temporales y de dominio para mantener la brevedad).

Topic/filter name	**Summarized Query (Field: TITLE-ABS-KEY)**
1. Self-Architecting Systems	("self-architecting system*" OR "auto-architecting system*" OR "self-architectural design" OR "auto-architectural design")
2. Self-Design	("self-design" OR "self design" OR "auto-design" OR "auto design" OR "self software design" OR "auto software design")
3. Evolutionary Design in Software Systems	(("evolutionary design" OR "evolutionary software design") AND ("software system*" OR "software engineering"))
4. Generative Design SW Systems	(("generative design" OR "generative software design") AND ("software system*" OR "software engineering"))
5. Software Design Automation	("software design automation" OR "design automation in software engineering" OR "automated software design" OR "AI for software design")
6. Self-Sufficient Systems	("self-sufficient system*" OR "autonomous self-sufficient software" OR "self-sufficient software system*")
7. AI for Software Engineering	(("AI for software engineering" OR "artificial intelligence in software engineering" OR "machine learning in software engineering"))
8. Self/Auto-Architectural Design	("self-architectural design" OR "auto-architectural design" OR "self adaptive architecture" OR "self-adaptive design")
9. Self/Auto-Interface Design	("self-interface design" OR "auto-interface design" OR "autonomous interface design" OR "adaptive interface design")
10. Self/Auto-Component Design	("self-component design" OR "auto-component design" OR "autonomous component design" OR "self-generated component*")
11. Self/Auto-Data S. Design	("self-data structure design" OR "auto-data structure design" OR "autonomous data structure" OR "self-optimizing data structure*")
12. Self/Auto-Algorithm Design	("self-algorithm design" OR "auto-algorithm design" OR "autonomous algorithm" OR "automated algorithm" OR "generative algorithm design" OR "AI-based algorithm")

Las consultas fueron ejecutadas de manera independiente y posteriormente los resultados fueron unificados (eliminando duplicados). El conjunto final de artículos formaron los *estudios iniciales seleccionados* de esta revisión, recordando que fueron filtrados mediante los criterios de inclusión y exclusión descritos a continuación.

5.1.1.3. Criterios de Inclusión y de exclusión

Para garantizar la calidad y relevancia de los estudios seleccionados, se establecen criterios explícitos de inclusión y exclusión de acuerdo al objetivo de esta revisión, la cual se centra en la automatización del diseño en el marco de los SES.

Criterios de inclusión

- **CI1: Periodo Temporal.** publicaciones desde **2015 hasta Marzo de 2026**. Un rango inicial (2005 – 2025) fue considerado, pero se actualizo para buscar técnicas y avances en el área de la Inteligencia Artificial, evadiendo la literatura del *computo autonómico* para enfocarnos unicamente en los sistemas de autoingeniería.
- **CI2: Idioma.** Solo artículos en **Ingles** fueron incluidos para alinearnos con el lenguajes mas adoptado en la divulgación científica en el mundo.
- **CI3: Disponibilidad y Claridad.** los estudios deben proveer **acceso completo** y una **metodología claramente definida** para permitir un análisis comprensivo y total de los mismo.
- **CI4: Dominio especifico.** Unicamente investigaciones en el campo de la **Ingeniería de software** que aborde de manera explicita o de manera indirecta cubra lo que es el auto-diseño de una manera unificada como proceso total o parcial.
- **CI5: Relevancia Científica.** Selección de estudios que han demostrado impacto en la comunidad científica a través de las citas, mientras se busca balancear este criterio, evaluando publicaciones reciente que no tienen un acumulado de citas pero si un gran contribución teórica o técnica en el tema.

Criterios de exclusión

Los siguientes criterios fueron usados para filtrar artículos que no coincidan con los requerimientos de la investigación:

- **CE1: Duplicación. publicaciones duplicadas** o **múltiples versiones** del mismo estudio; en cuyos casos, solo el mas completo o versión mas reciente sera considerada.

- **CE2: Tipo de Documento. Documentos incompletos**, **resúmenes extendidos**, o **publicaciones de posters** que carecen de suficiente detalle técnico para su evaluación.
- **EC3: Irrelevancia Disciplinaria.** estudios fuera del dominio de **Ingeniería de Software** sin un enlace directo al auto-diseño.
- **CE4: Deficiencias Metodológicas.** estudios que carezcan de **rigor metodológico**, particularmente lo que no tienen una clara descripción de las técnicas propuestas, algoritmos o conceptos.

Estos filtros (revisar Tabla 5.4) permiten la identificación de estudios representativos y de alta calidad, centrados en avances recientes en el ámbito conceptual y prácticos en el área de auto-diseño y auto-ingeniería, garantizando así la validez y relevancia de esta revisión sistemática de la literatura (este capitulo en particular).

Tabla 5.4: Resumen de los criterios de inclusión y exclusión

Tipo	**Criterio**
Inclusión	**CI1:** Publicaciones entre 2015 y 2025.
	CI2: Publicaciones en Ingles.
	CI3: Disponibilidad completa con metodología detallada.
	CI4: Centrado unicamente en software engineering, self-engineering, or self-design.
	CI5: Estudios relevantes con contribuciones recientes o numero de citas destacables.
Exclusión	**CE1:** Publicaciones duplicadas o múltiples versiones del mismo estudio.
	CE2: Resúmenes, posters o documentos incompletos.
	CE3: Estudios fuera del dominio de los sistemas de software.
	CE4: Trabajos con falta de detalles en su metodología o rigor científico.

5.1.1.4. Proceso de selección de estudios

Este bloque describe como fue la selección de estudios siguiendo las pautas de PRISMA. La Figura 5.1 ilustra el flujo general de identificación de estudios, cribado (aplicación de criterios de inclusión y exclusión), evaluación de elegibilidad (lectura del resumen y, posteriormente, del documento completo) e inclusión final. Los artículos semilla (conjunto inicial) se seleccionaron a partir de los estudios que cumplían los criterios de inclusión y obtuvieron las puntuaciones más altas en la evaluación de calidad durante la búsqueda inicial en bases de datos. Posteriormente, se identificaron estudios adicionales mediante la técnica de bola de nieve (hacia atrás y hacia adelante), siguiendo las directrices propuestas por Claes Wohlin [45]. Este proceso iterativo consistió en examinar las listas de referencias de los artículos semilla (bola de nieve hacia atrás) e identificar los artículos que los citaban (bola de nieve hacia adelante), aplicando los mismos criterios de inclusión y exclusión en cada iteración.

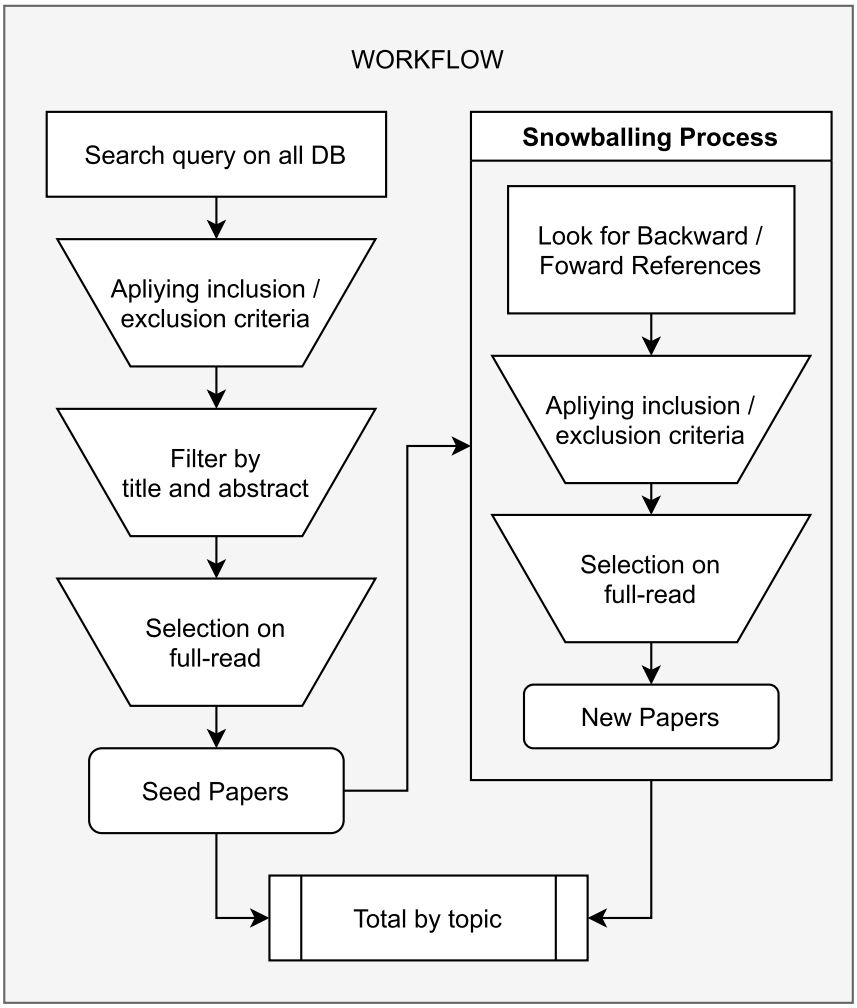


Figura 5.1: Revisión del flujo de trabajo basado en PRISMA.(Diagrama elaborado en Draw.io)

Para complementar el flujo de PRISMA, se añade una vista preliminar a manera de cuantificar los resultados, así detallar los los estudios seleccionados por cada etapa del proceso y especificando el tópico en particular. Esta representación permite comprender mejor cómo el conjunto inicial de estudios recuperados se refinó progresivamente hasta llegar a los estudios seleccionados finalmente y analizados en esta revisión. La Figura 5.2 resume este proceso mediante la especificación explicita del umero de artículos por cada etapa de selección.

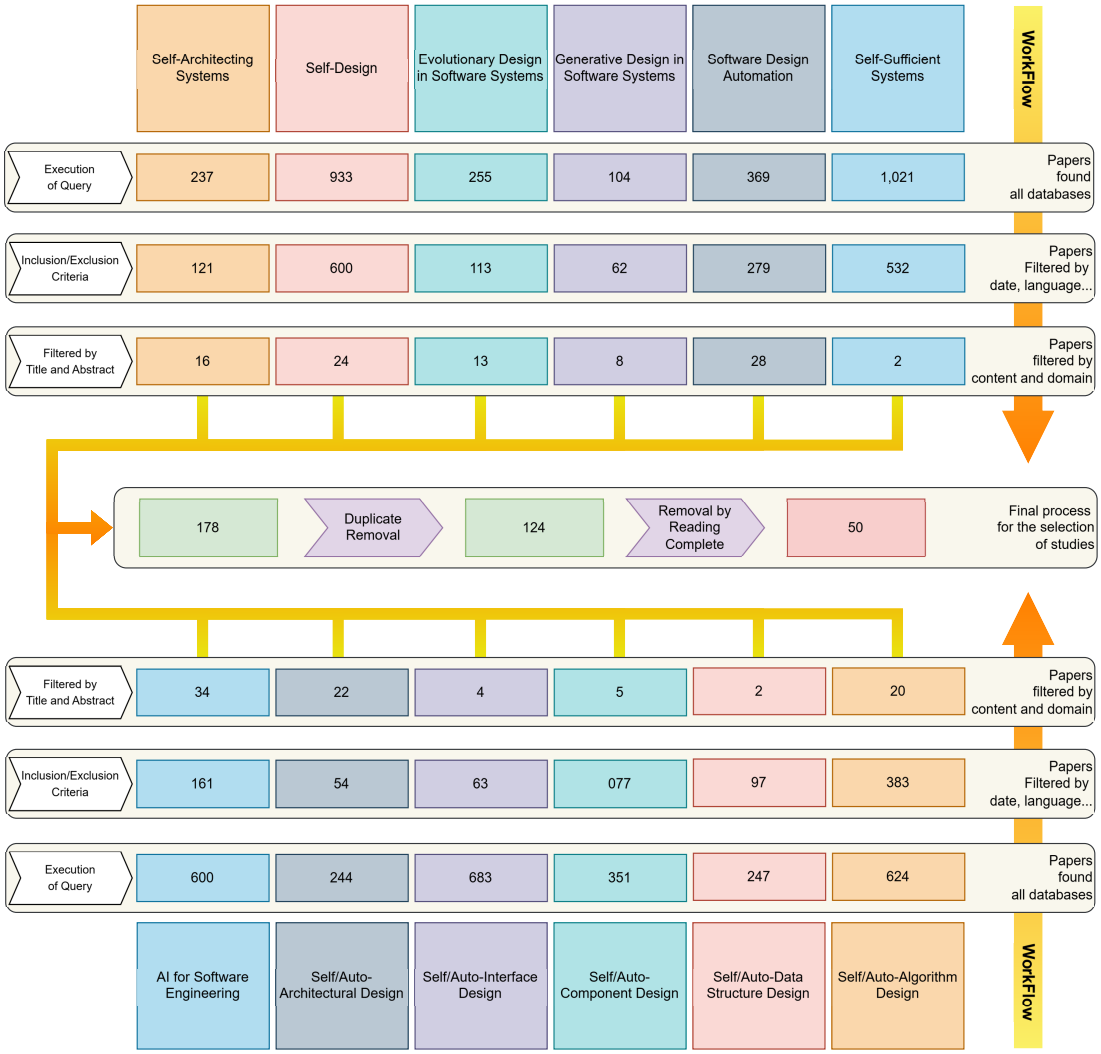


Figura 5.2: Proceso de selección de artículos con resultados cuantitativos por etapa de cribado. (Diagrama elaborado en Draw.io)

5.1.1.5. Extracción de datos

La extracción de datos se realizó exclusivamente a partir de los estudios que cumplían con los criterios de inclusión definidos y presentados en la Tabla 5.4. Este proceso se llevó a cabo de manera sistemática para garantizar la coherencia y la trazabilidad de la información recopilada de cada estudio seleccionado. Dada la heterogeneidad de la literatura analizada, se empleó un análisis narrativo para sintetizar la información. En este contexto, los enfoques analizados se organizaron en categorías temáticas derivadas de los objetivos de la revisión y de las preguntas de investigación formuladas. Esta estrategia permitió identificar patrones recurrentes, tendencias emergentes y áreas poco exploradas dentro de

los estudios seleccionados. Asimismo, la estructuración temática facilitó una evaluación comparativa de cómo cada estudio aborda el autodiseño en el contexto de la ingeniería de software, destacando las similitudes y diferencias entre los enfoques propuestos, así como el grado en que sus hallazgos contribuyen a responder a las preguntas de investigación planteadas en esta revisión.

5.1.1.6. Resultados

Los resultados completos están en el articulo titulado "Automated Software Design Towards Zero Human Intervention: A Systematic Literature Review" que actualmente esta sometido (en proceso de validación), pero para este trabajo se extrajeron los resultados mas importantes en el capítulo 3. La siguiente es una síntesis de lo que fue la discusión.

5.1.1.7. Discusión

La revisión revela un punto de inflexión hacia la automatización del diseño de software a través de cuatro paradigmas principales: la *refactorización basada en búsqueda (SBR)*, el *autodiseño algorítmico mediante mejora genética (GI)*, la *adaptabilidad formal con ontologías y planificación (PDDL)*, y la transición hacia la *Ingeniería de Software 3.0 (SE 3.0)* basada en ecosistemas multiagente. Aunque estos enfoques permiten manipular arquitecturas complejas y redefinen el rol del desarrollador hacia el de un orquestador estratégico, enfrentan importantes desafíos técnicos: la pérdida de semántica en los modelos matemáticos, la inestabilidad durante el reemplazo de código en caliente, la sobrecarga computacional en entornos inciertos y el riesgo de alucinaciones de diseño ante la falta de restricciones verificables y gobernanza en los agentes. Finalmente, se evidencia una brecha crítica entre el potencial teórico y la práctica, ya que la mayoría de los estudios se limitan a entornos controlados o sintéticos; por ello, se concluye que la integración del razonamiento simbólico con modelos de lenguaje es indispensable para lograr arquitecturas autogeneradas transparentes, funcionales y viables en entornos de producción reales.

5.2. Propuesta de un Sistema Multi-Agente (MAS) para el Auto-Diseño

5.2.1. Formulación General del Sistema

5.2.1.1. Definición del Entorno y Modelo de Auto-Diseño

5.2.1.2. Formalización Matemática del MAS

Sea $R = \{r_1, r_2, \ldots, r_n\}$ el conjunto de requerimientos del sistema y $D^* = D_A \cup D_C \cup D_{DS} \cup D_{ALG}$ el conjunto de artefactos de diseño producidos. El sistema multiagente se define como:

$$MAS = \{A_{arch}, A_{comp}, A_{ds}, A_{alg}\} \tag{5.1}$$

donde cada agente A_i opera bajo la función de transición BDI:

$$A_i : (B_i, P_i, G_i, S) \to (I_i, D_i) \tag{5.2}$$

5.2.2. Especificación del Sistema (System Specification)

5.2.2.1. Jerarquía de Metas (System Goals)

- **G1. Mantener la consistencia del diseño (D^*):**
  - **G1.1:** Analizar el impacto de cambios en requerimientos (R).
  - **G1.2:** Evaluar la necesidad de adaptación incremental vs. metamorfosis.
- **G2. Seleccionar alternativas óptimas por capa de diseño:**
  - **G2.1:** Determinar componentes e interfaces compatibles.
  - **G2.2:** Seleccionar estructuras de datos eficientes.
  - **G2.3:** Seleccionar algoritmos mediante búsqueda MCTS.

5.2.2.2. Interfaz con el Entorno (Percepts y Actions)

Percepciones (P): Notificación de cambios en requerimientos ($IN_{\text{requerimientos}}$), condiciones de carga operativa, alertas de incompatibilidad inter-agente.

Acciones (*Actions*): Generación/modificación de artefactos D_i, actualización del Estado de Diseño (S), emisión de solicitudes de reconsideración, exportación del documento final (OUT_{SDD}).

5.2.2.3. Escenarios de Uso Principales (Scenarios)

1. **Escenario Nominal de Diseño en Cascada:** Recepción de requerimientos → Modificación arquitectónica (A_{arch}) → Reconfiguración de componentes (A_{comp}) → Selección de datos (A_{ds}) → Búsqueda algorítmica (A_{alg}) → Generación de OUT_{SDD}.
2. **Escenario de Retroalimentación por Conflictos:** Detección de incompatibilidad en A_{alg} → Emisión de alerta de conflicto a A_{ds} / A_{comp} → Reconsideración de decisiones previas → Re-evaluación de consistencia global.

5.2.3. Diseño Arquitectónico (Architectural Design)

5.2.3.1. Identificación y Justificación de Agentes

- **Agente de Arquitectura (A_{arch}):** Alta cohesión en análisis de requerimientos y decisiones estructurales globales.
- **Agente de Componentes e Interfaces (A_{comp}):** Dominio sobre contratos de interacción y topología de módulos.
- **Agente de Estructuras de Datos (A_{ds}):** Dominio sobre esquemas de almacenamiento y accesos.
- **Agente de Algoritmos (A_{alg}):** Especialización en caracterización del problema y optimización algorítmica.

5.2.3.2. Estructura Global del Sistema (System Overview)

El flujo general de procesamiento y dependencia de decisiones se expresa como:

$$IN_{\text{requerimientos}} \rightarrow A_{arch} \rightarrow A_{comp} \rightarrow A_{ds} \rightarrow A_{alg} \rightarrow OUT_{SDD} \tag{5.3}$$

Todos los agentes interactúan leyendo y actualizando el Almacén de Datos Compartido que contiene el Estado Global de Diseño (S).

5.2.3.3. Protocolo de Negociación y Coordinación

El mecanismo de retroalimentación para la resolución de conflictos inter-agente se rige bajo la secuencia iterativa:

$$\text{Proponer } (A_i \to S) \;\to\; \text{Evaluar } (A_{i+1}) \;\to\; \text{Detectar Conflicto } (A_{i+1} \to A_i) \;\to\; \text{Reconsiderar} \tag{5.4}$$

5.2.4. Diseño Detallado BDI (Detailed Design)

5.2.4.1. Estructura BDI Interna de los Agentes

Agente	**Creencias** (B)	**Metas** (G)	**Planes / Intenciones** (I)
A_{arch}	Arquitectura actual, historial de cambios, catálogo de estilos.	Minimizar costo de metamorfosis, garantizar cobertura de R.	Plan de Adaptación Incremental, Plan de Metamorfosis Arquitectónica.
A_{comp}	Contratos de interfaz, métricas de carga del entorno.	Balancear carga, evitar incompatibilidad de interfaces.	Plan de Reconfiguración de Componentes, Plan de Adaptación de Conectores.
A_{ds}	Patrones de acceso, volumen de datos, restricciones de memoria.	Optimizar tiempo de acceso y uso de memoria.	Plan de Selección de Estructuras, Plan de Remapeo de Datos.
A_{alg}	Ontología de algoritmos, espacio de características P.	Encontrar algoritmo óptimo compatible.	Plan de Caracterización de Problema, Plan de Búsqueda MCTS.

Tabla 5.5: Especificación BDI de los agentes del MAS propuesta.

5.2.4.2. Caso de Estudio de Caja Blanca: Agente de Algoritmos (A_{alg})

El agente A_{alg} opera mediante un mecanismo explícito de razonamiento en tres fases:

1. **Caracterización del Problema:** Representado mediante la tupla:

$$P = (Objective, Interaction, Knowledge, Processing, Layer, Uncertainty) \tag{5.5}$$

2. **Búsqueda mediante MCTS sobre Ontología:** Búsqueda guiada en el árbol de soluciones donde cada expansión evalúa la compatibilidad con las restricciones impuestas por A_{arch}, A_{comp} y A_{ds}.

3. **Disparo de Eventos de Retroalimentación:** Si la rama MCTS no encuentra alternativas válidas, el agente genera un evento interno de conflicto que activa la re-evaluación en los agentes previos.

5.2.5. Análisis de Propiedades del Modelo de Agentes

- **Autonomía:** Cada agente evalúa sus intenciones sin control centralizadoizado, tomando decisiones en su dominio de diseño.
- **Reactividad:** Respuesta ante modificaciones en los requerimientos ($IN_{\text{requerimientos}}$) o alertas de incompatibilidad.
- **Orientación a Metas:** Comportamiento guiado por la preservación de la consistencia del diseño (D^*).
- **Interacción y Coordinación:** Intercambio de propuestas y reconsideraciones vía el protocolo de consenso.

Capítulo 6

Experimentación y Análisis de Resultados

6.1. Análisis de selección automática de algoritmos

6.1.1. Análisis de la arquitectura: auto-diseño

6.2. Experimentación

6.2.1. Caso de estudio: nuevo problema, nueva solución

6.2.2. Experimento: Identificación del Problema

6.2.3. Experimento: Identificación del Algoritmo

Capítulo 7

Conclusiones y Trabajo Futuro

7.1. Conclusiones

7.2. Objetivos logrados

7.3. Trabajo a futuro

Referencias Bibliográficas

[1] A. Ampatzoglou, A. Ampatzoglou, A. Chatzigeorgiou y P. Avgeriou, «The financial aspect of managing technical debt: A systematic literature review,» *Information and Software Technology*, vol. 64, págs. 52-73, 2015, ISSN: 0950-5849. DOI: https://doi.org/10.1016/j.infsof.2015.04.001 dirección: https://www.sciencedirect.com/science/article/pii/S0950584915000762

[2] W. K. G. Assunção, L. Marchezan, L. Arkoh, A. Egyed y R. Ramler, «Contemporary Software Modernization: Strategies, Driving Forces, and Research Opportunities,» *ACM Trans. Softw. Eng. Methodol.*, vol. 34, n.º 5, mayo de 2025, ISSN: 1049-331X. DOI: 10.1145/3708527 dirección: https://doi.org/10.1145/3708527

[3] G. P. Pérez Carrillo, «Nuevo Marco Conceptual de Sistemas de Autoingeniería y una Arquitectura para la Gestión de su Ciclo de Vida,» Tesis de Maestría en Ciencias, Centro de Investigación y de Estudios Avanzados del Instituto Politécnico Nacional Unidad Guadalajara, Guadalajara, México, ago. de 2024.

[4] G. Pérez-Carrillo, M. Siller y L. Alonso, «Self-Engineering Systems: A New Conceptual Framework and a Reference Meta-Architecture for its Life Cycle Management,» *Automated Software Engineering*, mayo de 2026. DOI: 10.21203/rs.3.rs-9577141/v1 dirección: http://dx.doi.org/10.21203/rs.3.rs-9577141/v1

[5] A. Smeda, M. Oussalah y T. Khammaci, «MADL: Meta Architecture Description Language,» en *Third ACIS Int'l Conference on Software Engineering Research, Management and Applications (SERA'05)*, 2005, págs. 152-159. DOI: 10.1109/SERA.2005.50

[6] I. Sommerville, *Software Engineering, Global Edition*, en, 10.ª ed. London, England: Pearson Education, jul. de 2015.

[7] I. Sommerville, *Software Engineering*, 9.ª ed. Boston: Addison-Wesley, 2011, ISBN: 978-0-13-703515-1.

[8] P. B. Kruchten, «The 4+1 View Model of Architecture,» *IEEE Softw.*, vol. 12, págs. 42-50, 1995. DOI: 10.1109/52.469759

[9] M. Carwehl, T. Vogel, G. Rodrigues y L. Grunske, «Runtime Verification of Self-Adaptive Systems with Changing Requirements,» *2023 IEEE/ACM 18th Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS)*, págs. 104-114, 2023. DOI: 10.1109/seams59076.2023.00024

[10] IEEE Computer Society, *Guide to the Software Engineering Body of Knowledge (SWEBOK)*, 4.ª ed. Los Alamitos, CA: IEEE, 2025, ISBN: 978-0-7695-5166-1. dirección: https://www.computer.org/education/bodies-of-knowledge/software-engineering

[11] «IEEE/EIA Guide - Industry Implementation of International Standard ISO/IEC 12207 : 1995 (ISO/IEC 12207) - Software Life Cycle Processes - Life Cycle Data,» *IEEE/EIA 12207.1-1997*, págs. 1-38, 1998. DOI: 10.1109/IEEESTD.1998.86377

[12] «ISO/IEC/IEEE International Standard - Systems and software engineering System life cycle processes,» *ISO/IEC 15288:2008(E) IEEE Std 15288-2008 (Revision of IEEE Std 15288-2004)*, págs. 1-84, 2008. DOI: 10.1109/IEEESTD.2008.6093923

[13] «IEEE Standard for Information Technology–Systems Design–Software Design Descriptions,» *IEEE STD 1016-2009*, págs. 1-35, 2009. DOI: 10.1109/IEEESTD.2009.5167255

[14] *Innovations in Multi-Agent Systems and Applications - 1*. Springer Berlin Heidelberg, 2010, ISBN: 9783642144356. DOI: 10.1007/978-3-642-14435-6 dirección: http://dx.doi.org/10.1007/978-3-642-14435-6

[15] Y. Zhang, M. Zhao, Y. Zhang e Y.-m. Cheung, «Trending Applications of Large Language Models: A User Perspective Survey,» *IEEE Transactions on Artificial Intelligence*, vol. 7, n.º 4, págs. 1835-1852, 2026. DOI: 10.1109/TAI.2025.3620272

[16] W. X. Zhao et al., «A Survey of Large Language Models,» *Frontiers of Computer Science*, vol. 20, n.º 12, mayo de 2026, ISSN: 2095-2236. DOI: 10.1007/s11704-026-60308-3 dirección: http://dx.doi.org/10.1007/s11704-026-60308-3

[17] T. Xiao y J. Zhu, *Foundations of Large Language Models*, 2026. arXiv: 2501.09223 [cs.CL]. dirección: https://arxiv.org/abs/2501.09223

[18] P. Shojaee, I. Mirzadeh, K. Alizadeh, M. Horton, S. Bengio y M. Farajtabar, «The Illusion of Thinking: Understanding the Strengths and Limitations of Reasoning Models via the Lens of Problem Complexity,» *arXiv preprint*, jun. de 2025. arXiv: 2506.06941 [cs.AI]. dirección: https://arxiv.org/abs/2506.06941

[19] M. Świechowski, K. Godlewski, B. Sawicki y J. Mańdziuk, «Monte Carlo Tree Search: a review of recent modifications and applications,» *Artificial Intelligence Review*, vol. 56, n.º 3, págs. 2497-2562, 2022, ISSN: 1573-7462. DOI: 10.1007/s10462-022-10228-y dirección: http://dx.doi.org/10.1007/s10462-022-10228-y

[20] T. R. Gruber, «A translation approach to portable ontology specifications,» *Knowledge Acquisition*, vol. 5, n.º 2, págs. 199-220, 1993, ISSN: 1042-8143. DOI: https://doi.org/10.1006/knac.1993.1008 dirección: https://www.sciencedirect.com/science/article/pii/S1042814383710083

[21] N. Guarino, D. Oberle y S. Staab, «What Is an Ontology?» En *Handbook on Ontologies*. Springer Berlin Heidelberg, 2009, págs. 1-17, ISBN: 9783540926733. DOI: 10.1007/978-3-540-92673-3_0 dirección: http://dx.doi.org/10.1007/978-3-540-92673-3_0

[22] I. Horrocks, P. F. Patel-Schneider y F. van Harmelen, «From SHIQ and RDF to OWL: the making of a Web Ontology Language,» *Journal of Web Semantics*, vol. 1, n.º 1, págs. 7-26, dic. de 2003, ISSN: 1570-8268. DOI: 10.1016/j.websem.2003.07.001 dirección: http://dx.doi.org/10.1016/j.websem.2003.07.001

[23] L. Duran Polanco, «Selección de algoritmos para la toma de decisiones en el IoT basada en un modelo semántico,» Tesis de Maestría, Centro de Investigación y de Estudios Avanzados del Instituto Politécnico Nacional, Unidad Guadalajara, Guadalajara, Jalisco, México, feb. de 2026.

[24] L. Durán-Polanco y M. Siller, «A taxonomy for decision making in IoT systems,» *Internet of Things*, vol. 24, pág. 100 904, 2023, ISSN: 2542-6605. DOI: https://doi.org/10.1016/j.iot.2023.100904 dirección: https://www.sciencedirect.com/science/article/pii/S2542660523002275

[25] IBM, *Autonomic Computing: IBM's Perspective on the State of Information Technology*, White Paper, Armonk, NY, 2001. dirección: http://www.ibm.com/research/autonomic

[26] J. O. Kephart y D. M. Chess, «The vision of autonomic computing,» *Computer*, vol. 36, n.º 1, págs. 41-50, 2003.

[27] IBM Corporation, «Autonomic Computing: An Architectural Blueprint for Autonomic Computing,» IBM, White Paper, jun. de 2006.

[28] B. Porter, P. Faulkner Rainford y R. Rodrigues-Filho, «Self-Designing Software,» *Commun. ACM*, vol. 68, n.º 1, págs. 50-59, dic. de 2024, ISSN: 0001-0782. DOI: 10.1145/3678165 dirección: https://doi-org.access.biblioteca.cinvestav.mx/10.1145/3678165

[29] S. Idreos et al., «Learning Data Structure Alchemy,» eng, *Bulletin of the IEEE Computer Society Technical Committee on Data Engineering*, vol. 42, págs. 46-57, 2019.

[30] S. Idreos et al., «The Periodic Table of Data Structures.,» *IEEE Data Eng. Bull.*, vol. 41, n.º 3, págs. 64-75, 2018.

[31] C. Loncaric, «Data structure synthesis,» en *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering*, ép. FSE 2016, Seattle, WA, USA: Association for Computing Machinery, 2016, págs. 1073-1075, ISBN: 9781450342186. DOI: 10.1145/2950290.2983946 dirección: https://doi.org/10.1145/2950290.2983946

[32] C. Loncaric, M. D. Ernst y E. Torlak, «Generalized data structure synthesis,» en *Proceedings of the 40th International Conference on Software Engineering*, ép. ICSE '18, Gothenburg, Sweden: Association for Computing Machinery, 2018, págs. 958-968, ISBN: 9781450356381. DOI: 10.1145/3180155.3180211 dirección: https://doi.org/10.1145/3180155.3180211

[33] Q. Zhao, T. Liu, B. Yan, Q. Duan, J. Yang e Y. Shi, «Automated Metaheuristic Algorithm Design With Autoregressive Learning,» *IEEE Transactions on Evolutionary Computation*, vol. 29, n.º 5, págs. 2004-2018, oct. de 2025, ISSN: 1941-0026. DOI: 10.1109/tevc.2024.3464677 dirección: http://dx.doi.org/10.1109/TEVC.2024.3464677

[34] W. Yi, R. Qu y L. Jiao, «Automated algorithm design using proximal policy optimisation with identified features,» *Expert Systems with Applications*, vol. 216, pág. 119 461, abr. de 2023, ISSN: 0957-4174. DOI: 10.1016/j.eswa.2022.119461 dirección: http://dx.doi.org/10.1016/j.eswa.2022.119461

[35] A. Ramírez, J. R. Romero y S. Ventura, «An approach for the evolutionary discovery of software architectures,» *Information Sciences*, vol. 305, págs. 234-255, 2015, ISSN: 0020-0255. DOI: https://doi.org/10.1016/j.ins.2015.01.017 dirección: https://www.sciencedirect.com/science/article/pii/S0020025515000559

[36] T. Houichime e Y. El Amrani, «Optimized design refactoring (ODR): a generic framework for automated search-based refactoring to optimize object-oriented software architectures,» *Automated Software Engineering*, vol. 31, n.º 2, jun. de 2024, ISSN: 1573-7535. DOI: 10.1007/s10515-024-00446-9 dirección: http://dx.doi.org/10.1007/s10515-024-00446-9

[37] H. K. Dam, «Towards Multi-Agentic AI for automated software design and modelling: challenges and opportunities,» en *2025 40th IEEE/ACM International Conference on Automated Software Engineering Workshops (ASEW)*, IEEE, nov. de 2025, págs. 311-314. DOI: 10.1109/asew67777.2025.00063 dirección: http://dx.doi.org/10.1109/ASEW67777.2025.00063

[38] R. Li et al., «Maad: Automate software architecture design through knowledge-driven multi-agent collaboration,» *arXiv preprint arXiv:2507.21382*, 2025.

[39] V. Rajendran, D. Besiahgari, S. C. Patil, M. Chandrashekaraiah y V. Challagulla, «A Multi-Agent LLM Environment for Software Design and Refactoring: A Conceptual Framework,» en *SoutheastCon 2025*, IEEE, mar. de 2025, págs. 488-493. DOI: 10.1109/southeastcon56624.2025.10971563 dirección: http://dx.doi.org/10.1109/SOUTHEASTCON56624.2025.10971563

[40] F. J. Affonso, G. Nagassaki Campos y G. Guiguer Menaldo, «A Reference Architecture Based on Reflection for Self-Adaptive Software: A Second Release,» *IEEE Access*, vol. 12, págs. 97 476-97 499, 2024, ISSN: 2169-3536. DOI: 10.1109/ACCESS.2024.3428368

[41] D. Weyns y J. Andersson, «From Self-Adaptation to Self-Evolution Leveraging the Operational Design Domain,» en *2023 IEEE/ACM 18th Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS)*, mayo de 2023, págs. 90-96. DOI: 10.1109/SEAMS59076.2023.00022

[42] M. J. Page et al., «The PRISMA 2020 statement: an updated guideline for reporting systematic reviews,» *BMJ*, vol. 372, 2021. DOI: 10.1136/bmj.n71 eprint: https://www.bmj.com/content/372/bmj.n71.full.pdf. dirección: https://www.bmj.com/content/372/bmj.n71

[43] B. Kitchenham y S. Charters, «Guidelines for performing Systematic Literature Reviews in Software Engineering,» vol. 2, ene. de 2007.

[44] M. Gusenbauer y N. R. Haddaway, «Which academic search systems are suitable for systematic reviews or meta-analyses? Evaluating retrieval qualities of Google Scholar, PubMed, and 26 other resources,» *Research Synthesis Methods*, vol. 11, n.º 2, págs. 181-217, ene. de 2020, ISSN: 1759-2887. DOI: 10.1002/jrsm.1378 dirección: http://dx.doi.org/10.1002/jrsm.1378

[45] C. Wohlin, «Guidelines for snowballing in systematic literature studies and a replication in software engineering,» en *Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering*, ép. EASE '14, ACM, mayo de 2014, págs. 1-10. DOI: 10.1145/2601248.2601268 dirección: http://dx.doi.org/10.1145/2601248.2601268

Apéndice A

Código Fuente Relevante